Università degli Studi di Milano
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI

CORSO DI LAUREA MAGISTRALE IN
INFORMATICA

REPRODUCTION AND ANALYSIS OF REAL-WORLD USE-AFTER-FREE ATTACKS AND STUDY OF DEFENCE TECHNIQUES

Supervisor: Prof. Andrea Lanzi
Co-supervisor: Dr. Andrea Monzani

Thesis by:

Stefano Nava

Matr. No. 27436A

Academic Year 2025-2026

Contents

Chapter 1

Introduction

Software security is nowadays a fundamental aspect of the development and maintenance of IT systems. The increasing complexity of applications, combined with the growing interconnection between devices and services, has significantly expanded the attack surface that can be exploited by potential malicious actors. Operating systems, browsers, drivers and distributed services are critical components whose proper functioning is closely linked to the robustness of their security properties.

In this context, proper memory management plays a central role, particularly in systems developed using low-level languages such as C and C++, which are defined as non-memory-safe. Although these languages offer high performance and control over resources, they are particularly prone to memory management errors, which can result in exploitable vulnerabilities. Memory-safe languages, in fact, have automatic mechanisms for controlling the allocation and deallocation of memory blocks. In the absence of these, while memory management operations are faster, an important safeguard for security is lost. Despite decades of research and the development of numerous mitigation techniques, such vulnerabilities continue to arise even in widely used and mature software [1, 2, 3], highlighting how memory security remains an ongoing challenge in the contemporary cybersecurity landscape. A particularly relevant example of this challenge is the Linux kernel, one of the most widely deployed and scrutinized pieces of software in existence, yet still subject to memory safety vulnerabilities. Given the complexity of its subsystems and the performance constraints under which it operates, the kernel cannot rely on garbage collection or other automatic memory management strategies [4]. Instead, it largely depends on reference counting [5], a technique by which each kernel object maintains a counter tracking the number of active references to it, with deallocation occurring only when the counter reaches zero. While reference counting provides a structured and efficient mechanism for controlling object lifetimes, it is not immune to subtle concurrency bugs: incorrect counter manipulations, race conditions on shared objects, or improper ordering of decrement and deallocation operations can all lead to use-after-free (UAF) vulnerabilities [6], in which a memory region is accessed after it has been freed and potentially

reused for a different purpose.

The objective of this thesis is to investigate whether static and dynamic analysis techniques can reliably detect UAF vulnerabilities arising specifically from reference counting errors in the Linux kernel, a class of bugs that, as will be discussed, has proven difficult to surface using existing automated tools. To this end, we analyze CVE-2025-21756 [7], a real-world vulnerability affecting the vsock subsystem [8], as a case study: we examine why it eluded prior detection, and what properties of the bug and of the analysis tools contribute to this blind spot.

The thesis is organized as follows:

- **Background**: it covers the fundamentals of memory management: stack and heap organization, pointer semantics and reference counting mechanisms; it introduces the concept of use-after-free vulnerabilities and their relationship with reference counting errors and discusses the Linux kernel architecture with particular attention to how UAF vulnerabilities manifest in that context. The chapter concludes with an overview of existing analysis tools for UAF detection, with a focused examination of CountDown [9] and UAFX [10], the two tools that we tested, central to this work;
- **CVE-2025-21756**: after a general overview of the vulnerability, it describes the vsock subsystem and its internal architecture, examines the test case provided by the kernel maintainers, and analyzes the root causes of the bug and the kernel functions involved;
- **CountDown testing - dynamic analysis**: this chapter documents the CountDown configuration phase and the tests performed. We first tested an unguided run and then a run with a starting corpus to direct the fuzzer toward the functions responsible for the UAF bug under investigation;
- **UAFX testing - static analysis**: similar to the chapter about CountDown, this one documents the experimental procedure conducted using UAFX: it covers the environment setup, the compilation of the relevant kernel modules, the extraction of LLVM [11] bitcode and the execution of the analysis, discussing the results obtained and their implications;
- **Conclusions**: in the final chapter, we summarize the results obtained from the experiments (both with CountDown and with UAFX), briefly discuss the challenges associated with vulnerability analyses of this type and specifically address the tools we examined. Finally, we conclude with some hypotheses regarding possible future research directions in this specific area.

Chapter 2

Background

2.1 Memory structure

Memory is one of the most critical components from a security point of view in a computer system, since during program execution it contains not only the data being processed but also the information necessary to determine the program's behavior. This includes:

- **variables** containing temporary data used by the executed program(s);
- **data structures**, relationships between different types of objects for the purpose of managing more complex data;
- **pointers with memory addresses** which tell the program "where" a certain resource is. They can point:
  - to other objects, to allow the program to read or write data;
  - to functions, to allow the program to execute instructions located at a specific position;
- **information used to manage the execution flow**, such as the address of the first instruction that needs to be executed when returning from a function call.

Unauthorized access to or arbitrary modification of these elements can therefore have consequences ranging from simple data corruption or program crashes to changes in the control flow and, in the most severe cases, the execution of unauthorized operations, which is the subject we're going to examine. To understand the origin and effects of the main memory corruption vulnerabilities, it is therefore useful to first consider how the memory space associated with a program is organized into its main regions.

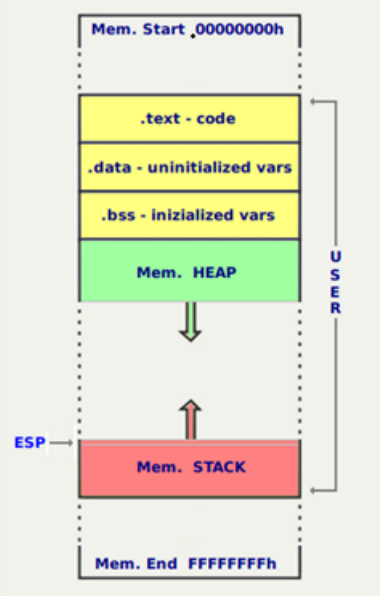


Figure 1: Structure of memory areas.

The two areas primarily affected by attacks of this kind are the stack and the heap.

2.1.1 Stack

The stack is a data structure organized according to a LIFO (Last In, First Out) policy and used primarily for managing function calls. Each function call involves the creation of a *stack frame* containing local variables, parameters and return information (such as the return address, which tells the program the point in the calling code from which to resume execution once the function has been exited). The allocation and deallocation of memory on the stack are automatic and handled by the compiler: when a function terminates, its corresponding frame (figure 2) is removed.

2.1.2 Heap

The heap is a memory region used for dynamic allocations, managed explicitly by the programmer using primitives such as `malloc`/`free` in C or `new`/`delete` in C++. These functions do not normally request a new memory region from the operating system for every individual location, but a user-space memory allocator manages larger regions of memory, dividing them into smaller blocks and keeping track of which portions are currently allocated or available for reuse.

On Linux systems, the allocator can obtain additional virtual memory from the kernel through mechanisms such as `brk()`, which changes the end of the process data segment,

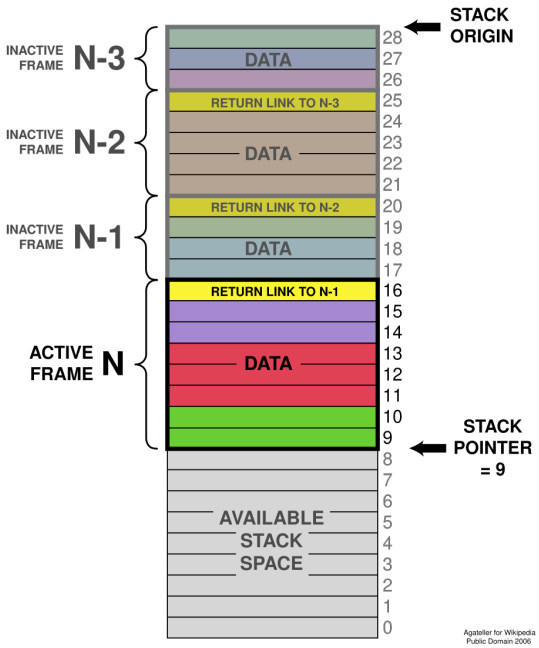


Figure 2: Example of a stack frame structure.

or `mmap()`, which creates new memory mappings. Large allocations are commonly serviced through `mmap()` while other allocations may be satisfied from memory already managed by the allocator or from an enlarged process heap. The exact strategy is implementation-dependent and may change according to allocation size and configuration [12, 13].

Unlike the stack, therefore, heap allocations do not follow a strict LIFO (Last In, First Out) order: allocated blocks may be released independently and subsequently reused for other object, which is essential for dynamically sized and long-lived data structures, but it also makes object lifetime and memory reuse more difficult to manage correctly, creating the conditions in which errors like UAF may appear.

2.1.3 Pointers

In memory-unsafe languages, pointers are identifiers that may contain the address of a specific memory area (to which they "point") and, for example in C, are initialised using the `malloc` function and freed using the `free` function. The programmer can use them to perform operations both on the addresses they contain and on the values contained in the memory cells they point to.

Given their nature, they are one of the main vectors for memory-related attacks. We therefore introduce the concept of a **dangling pointer** [14]: a pointer, if manipulated incorrectly,

may continue to point to an area that is no longer valid, typically because it has been deallocated or has expired. This allows an attacker to read or write content to which they should not have access.

Listing 2.1: Example of dangling pointer in C.

```c
void dangling_pointer_example(void){
    int *ptr = (int *)malloc(sizeof(int));
    *ptr = 5;
    free(ptr); // ptr is now a dangling pointer
    ptr = NULL; // now it doesn't point to the freed zone anymore
}
```

Under the right conditions, exploiting a dangling pointer can lead to **privilege escalation** [15], i.e. a low-level user gaining administrator privileges, thereby putting sensitive data or, in the most serious cases, entire systems at risk.

2.1.4 Reference counting mechanisms

To mitigate the problem of dangling pointers and to avoid placing the responsibility for memory management directly on the programmer, **reference counting** [16] mechanisms have been introduced. These mechanisms associate each object with a counter that tracks the number of active references to it, so that, provided there are no errors, the relevant memory area is freed when it is no longer in use.

Listing 2.2: Simple example of a reference counting mechanism.

```c
void reference_count_example(void) {
    int *ptr = malloc(sizeof(int));
    int ref_count = 1;       // starting with 1 existing reference

    ref_count++;             // new reference
    ref_count--;             // release of a reference
    ref_count--;             // release of the last reference

    if (ref_count == 0)
        free(ptr);
}
```

When a new structure gains access to the object, the counter is incremented. Conversely, when the reference is no longer needed, the counter is decremented. When the counter reaches the

value *zero*, the system considers the object to be no longer in use and frees up the memory it occupies.

This approach helps to avoid premature deallocations and to coordinate concurrent access to shared resources more reliably. However, errors in counter management (such as multiple decrements or missing increments) can lead to critical conditions.

Early deallocation of an object that is still in use can, in fact, result in *dangling pointers* and use-after-free vulnerabilities, while failure to release references can cause memory leaks and uncontrolled memory consumption. For this reason, reference counting is one of the most delicate mechanisms in memory management within memory-unsafe systems.

2.2 Use-after-free (UAF)

2.2.1 What causes a UAF?

Every system has flaws and is vulnerable to attacks of varying severity. These flaws, which often stem from logical errors or the incorrect management of the lifecycle of objects in memory, can enable an attacker to compromise the integrity of the system, with different objectives. Some of the most known are **buffer overflow** [17] (a process uses a memory area that hasn't been allocated to it) or **double free** [18] (a memory area is deallocated two times corrupting the structures of the memory allocator).

The type of error we're going to focus on is **use-after-free** [6]. Incorrect management of the dynamic memory lifecycle can lead to access to portions of memory that have already been deallocated, allowing an attacker to reuse these areas to inject controlled data and influence the program's behaviour via dangling pointers.

Listing 2.3: Simple example of a UAF exploit in C.

```c
void uaf_example(void) {
    int *ptr = (int *)malloc(sizeof(int));
    *ptr = 5;

    free(ptr);        // ptr is now a dangling pointer

    *ptr = 10;        // use-after-free: access to freed memory

    ptr = NULL;       // the pointer is "neutralized"
}
```

A use-after-free vulnerability is fundamentally a violation of the expected lifetime of an object, which passes through three conceptual phases: allocation, usage and deallocation. During the

usage phase, one or more parts of the program can hold references to the object and assume that the memory area remains valid. Once the object is deallocated, this isn't true anymore and every reference that could still be used (*dangling pointers*, as explained in section 2.1.3) should be removed to avoid producing use-after-free conditions.

It's important to distinguish the lifetime of an *object* from the one of a *pointer* referring to it. When the `free()` function is called, the allocator marks the memory region as available for future use, but it doesn't remove the existing references to it, which could still be used. In large systems such as the Linux kernel, this distinction becomes particularly relevant because references to the same object may be held by different execution contexts. For example, one thread may remove and free an object from a shared data structure while another thread still retains a pointer to it. If the object's lifetime is not correctly synchronized across these contexts, the remaining reference becomes a dangling pointer and may subsequently lead to a use-after-free.

As mentioned in section 2.1.4, a solution to this problem is counting references for the objects, but an early deallocation can still result in a dangling pointer and can therefore be a specific (but not the only) cause of a use-after-free bug (and a potential exploit as a result).

2.2.2 Theory of refcount-based UAF exploitation

From an exploitation perspective, a reference-counting error becomes security-relevant when it causes an object to be deallocated while a reference to it remains reachable. The resulting dangling pointer does not by itself provide an attacker with a useful primitive: exploitation generally requires controlling, or at least influencing, what happens to the freed memory and how the stale reference is subsequently used.

A generalized exploitation process can be represented by the following sequence of steps:

1. Triggering an incorrect reference counting transition
2. Maintaining a reachable dangling pointer
3. Reusing the freed memory with a suitable replacement object
4. Accessing the new contents via the obsolete type
5. Transforming the resulting behaviour into a useful (usually malicious) memory primitive

The first stage of an exploit generally involves forcing an incorrect transition in the lifecycle. Depending on the vulnerability, the attacker may need to trigger an error-handling path, repeatedly register and unregister a resource, close an object while it is still in use by another operation, or coordinate multiple concurrent operations. In vulnerabilities relating to reference counting, the key event is often the premature release of the last reference recognised

by the system, even though another reachable reference still exists. The attacker must therefore maintain a code path capable of accessing the object after it has been deallocated, while simultaneously causing the counter to reach zero.

This phase can be particularly complex in concurrent systems. One thread might acquire or retain a pointer to an object, while another thread removes the object from a shared structure and frees what the system considers to be the last reference. If the acquisition of the reference is not correctly synchronised with the destruction of the object, we could end having a deallocation between the retrieval of the pointer and the increment of the counter. Similarly, a callback or an asynchronous task may retain an address without actually holding a reference to it. An attacker could attempt to widen these race condition windows by increasing the system's load or using multiple threads.

Once the object has been freed, the dangling pointer alone gives us only limited control. The next objective is to trick the allocator into reusing the freed memory for a new allocation whose contents can be controlled by the attacker (= **heap grooming**, also known as **heap feng shui** or **heap shaping** [19]). The attacker performs a controlled sequence of allocations and deallocations to manipulate the state of the heap and increase the likelihood that a chosen object will occupy the same memory region as the vulnerable object.

The feasibility of this replacement depends heavily on the behaviour of the allocator. Allocators, whether in kernel space or user space, tend to group allocations into size classes and to reuse freed regions for compatible objects. Consequently, the attacker typically seeks a replacement object belonging to the same allocation class as the vulnerable structure and which can be created repeatedly via an accessible interface.

Following memory reuse, two logically distinct objects occupy the same region at different times. The vulnerable code still interprets the dangling pointer according to the layout and semantics of the original object (as it should), while the allocator treats the region as belonging to the new object. This creates a form of type confusion induced by memory reuse: the fields of the new object may be interpreted as flags, sizes, reference counters, pointers to lists or function pointers belonging to the original structure.

The resulting access can provide various exploit primitives. A read via the dangling pointer may reveal heap addresses, kernel pointers or other sensitive values, making it easier to bypass mechanisms such as ASLR (Address Space Layout Randomization [20]: a technique for randomizing addresses of stack, heap and libraries in memory which usually helps preventing memory attacks). If a corrupted field is subsequently used as a pointer or index, we can obtain out-of-bounds read or arbitrary memory access primitives. A write via the obsolete reference, on the other hand, can alter the state of the new object, resulting in a controlled or partially controlled write to another area of memory.

Objects that contain pointers to functions or references to related structures are of particular interest to us, as their corruption can indirectly affect the control flow. However, modern exploits do not necessarily aim to immediately overwrite a function pointer. Mechanisms such

as Control-Flow Integrity (CFI) [21], memory non-executability and restrictions on code execution in kernel space often make direct redirection difficult. Instead, the exploit uses the initial vulnerability to construct more powerful read and write primitives, obtain address information and modify sensitive data structures. These primitives can then be exploited to alter credentials, permissions or the state of a process, depending on the execution context.

A pending reference may also interact directly with the reference counting mechanism. If the obsolete pointer is used in a deallocation operation, the system may decrement a field that no longer represents the original counter. This may corrupt the new object or cause a second deallocation of the same memory region. In such cases, the initial error may lead to additional conditions such as double-free or premature destruction of the new object. The exploit may therefore involve multiple consecutive violations of the object's lifecycle.

The practical exploitability of a reference-counting UAF therefore depends on several conditions: the attacker must be able to trigger the incorrect lifetime transition, preserve and subsequently use the dangling reference, obtain sufficiently predictable memory reuse and transform the resulting stale access into a useful primitive despite the allocator's behaviour and the mitigations provided by the target system. These requirements make real-world exploitation substantially more complex than the underlying reference-counting error itself.

Concrete examples of this exploitation process can be found in vulnerabilities such as CVE-2021-0920 [22] and CVE-2025-21756 [23], which illustrate how reference-counting and object-lifetime errors can be transformed into exploitable use-after-free conditions in real systems.

2.3 Linux and its kernel

2.3.1 The kernel

The kernel is the central component of a Linux operating system and operates with higher privileges than normal programs running in user space. Its main task is to mediate access to hardware resources and to provide applications with a controlled set of services via system calls. This is convenient for the developer, but it also carries some risks: while a bug in a user space program is confined to its process, memory corruption in kernel space can compromise the stability of the entire system or allow operations to be carried out with elevated privileges (the *privilege escalation* that we mentioned in section 2.1.3).

In addition to this, Linux keeps a highly modular structure, as some functionality can be compiled as a loadable module.

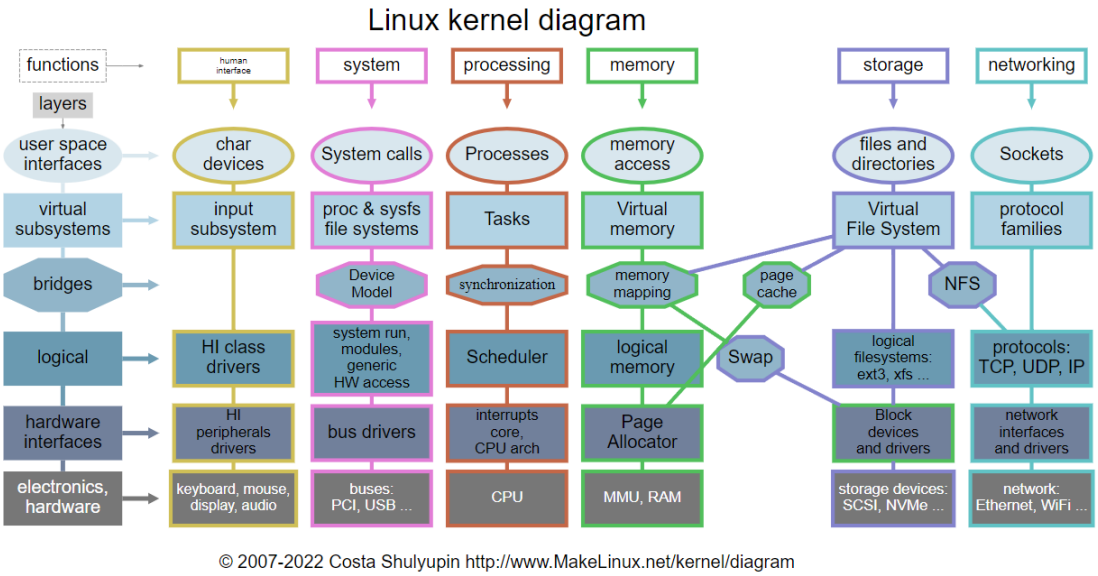


Figure 3: Diagram representing the modular structure of the Linux Kernel.

However, this flexibility does not provide complete isolation: a loaded module continues to run in kernel space and can access the kernel's internal structures, with the same aforementioned security implications.

Since its first release in 1991, Linux has evolved from a small-scale project into a platform used on a vast number of servers and devices all around the world, as we can see on the following table which sums up the distribution of Linux-powered machines around the world [24]:

Metric	**Linux Share**
Server OS Market (2026 est.)	51.3%
Websites (known OS)	61.3%
Top 1 Million Web Servers	96.3%
Government Data Centers	64.9%
Top500 Supercomputers	100%

This evolution has been accompanied by the introduction of numerous hardware architectures, new drivers and subsystems, which support a broader quantity of features but bring more security and stability issues on the kernel's side. The interfaces exposed to user space are kept as stable as possible, while the kernel's internal APIs may change significantly between different versions.

2.3.2 Kernel objects and memory management

Unlike operating systems such as Windows, where the concept of *kernel object* is formally defined and exposed through a unified object manager [25], the Linux kernel does not adopt a single, centralized abstraction for its internal entities but takes advantage of a rich set of well-defined C structures that represent the fundamental components of the system (e.g. processes, files, network sockets, memory regions). Each of these structures encapsulates both the data describing the entity and the state required to manage its lifecycle.

A representative example is `task_struct` [26]: this structure is used by the kernel to represent a process or thread. It contains hundreds of fields: scheduling information, memory maps, open file descriptors, credentials, and more; it is one of the most referenced structures throughout the kernel codebase. Similar in spirit are structures such as `struct socket` [27] and `struct sock` [28] in the networking subsystem, `struct inode` [29] for filesystem objects, or `struct sk_buff` [30] for network packets.

These objects are dynamically allocated on the kernel heap through the *slab allocator* [31] (in its modern SLUB implementation [32]), a memory management subsystem optimized for the frequent allocation and deallocation of fixed-size objects. The slab allocator maintains per-type caches of pre-allocated memory, reducing fragmentation and allocation overhead but also, as will be discussed, creating conditions that are relevant to the exploitation of memory corruption vulnerabilities.

Since many of these objects can be referenced simultaneously from multiple kernel subsystems (e.g. a socket can be held by a process, referenced by a pending timer, and pointed to by an active network connection at the same time) their lifetime cannot be tied to a single owner and this is where the reference counting mechanism comes into play. It is efficient, but its correctness relies entirely on every code path that acquires or releases a reference doing so properly. As the kernel grows in complexity and concurrency, maintaining this invariant across all subsystems becomes increasingly difficult and subtle errors in reference counter management can cause the counter to reach zero prematurely, triggering the deallocation of an object that is still reachable from some other part of the kernel.

2.3.3 Use-after-free vulnerabilities in the Linux kernel

Since kernel code executes with high privileges, consequences of reference-counting errors are generally more severe than in a normal user-space program. Detecting these vulnerabilities is complicated by the structure of the kernel itself. The allocation, use and release of an object may occur in different context and the relevant sequence of operation is likely not visible within a single control-flow path.

These characteristics make automated kernel analysis particularly complex. Detection tools have to deal with code spread across multiple modules that interact in a non-linear manner. Their analysis may also be constrained to a specific kernel version or compiler, or require modifications to the build system and, in some cases, full kernel instrumentation. Moreover,

the continuous evolution of internal APIs can quickly lead to compatibility issues if the analyzer is not actively maintained. The kernel documentation distinguishes between numerous categories of tools, such as:

- *static analyzers*: code analysis by tracking objects' paths without executing the code. An example is UAFX [10], which we tested and discuss in section 2.4.2
- *sanitizers*: suspect behaviour detection by instrumentating the code through a compiler
- *code coverage tools*: measurement of the quantity of code covered by tests
- *fuzzing tools*: generation of random inputs in order to test different execution paths for the analyzed software. An example is CountDown
- *mechanisms for detecting race conditions*: identification of critical code sections where a race condition may happen

LLVM

Most of the tools we have considered are based on LLVM [11], a modular infrastructure for building compilers and code analysis tools. Its central component is an intermediate representation (generated by a compiler; one of the most used is Clang [33]), commonly referred to as LLVM IR, on which modifications can be made that are independent of the source language and, to some extent, of the target architecture.

The use of LLVM is significant in the field of security because it allows analyses and transformations to be incorporated directly into the compilation process (e.g. by adding checks that will be executed at runtime). This integration enables more in-depth analyses than simply processing the source code textually, albeit at the expense of the performance of the system being analysed; however, a system compiled with LLVM will likely not be made available to the end user but will be used solely for analysis purposes.

2.4 Existing tools for detecting UAF vulnerabilities

For all known bugs, there is a wide range of tools described in the academic literature, used either individually or in combination, that serve (or help) to detect them. UAF issues, however, especially when related to reference counting problems, are particularly challenging. Due to the unpredictable nature of these bugs, the numerous conditions required to exploit them, and the variety of contexts in which they can occur, identifying them is by no means easy. In addition to the identification process itself, other problems may arise, such as large volumes of false positives that can cause a tool to be inaccurate on its own and require human review to distinguish them from true positives, or the need for extreme computational power to run the tool effectively.

In table 1 are some examples of tools capable of analyzing code and detecting use-after-free vulnerabilities. Since each vulnerability exploits different weaknesses, we have compiled a list of tools with different purposes and modes of operation to provide a broader range of options in this regard.
In the next sections, we will describe the tools that are relevant for this work. There were two tools specifically designed to detect UAF bugs in the Linux kernel: CountDown and CID. Unfortunately, CID does not have publicly available source code, and, as with other tools that do not meet this criterion, we decided to exclude them from the selection.
Since our focus was on a use-after-free vulnerability involving a reference counting issue, we also excluded *some* tools whose papers clearly stated that they would not be effective for this purpose.
Finally, we excluded the tools that would not have worked on the kernel because they were designed for smaller, less complex sections of code.
After considering these factors, the decision was made to choose CountDown and UAFX.

2.4.1 CountDown

CountDown is a fuzzer for the Linux kernel, presented at CCS 2024 [9] and it's specifically designed to detect memory timing errors caused by incorrect handling of reference counters. Unlike traditional fuzzers, which are primarily driven by code coverage, CountDown uses operations on reference counters as feedback to identify sequences of system calls that could potentially alter the lifecycle of those same objects. The tool is built on top of Syzkaller [35] (an open source, unsupervised, coverage-guided kernel fuzzer developed by Google) and extends its strategies for generating and mutating test programs.

During execution, CountDown instruments the kernel to associate increments, decrements, and accesses to reference counters with the system calls that triggered them. Based on this information, it establishes relationships between system calls that operate on the same counters and assigns them a higher probability of being combined in the generated tests. The tool can also repeatedly insert operations that decrement a counter, in an attempt to trigger the object's deallocation, followed by operations that continue to access it. In this way, fuzzing is directed toward sequences that can more effectively expose UAFs caused by incorrect operations on counters.

CountDown's approach is based on the idea of refcount-driven fuzzing to achieve two macro-objectives:

- *Identification of refcount-based relationships between syscalls*: instead of relying solely on syscall signatures or coverage, CountDown correlates syscalls that operate on the same object (*shared refcounts*).

- *Systematic exposure of UAF bugs*: through targeted mutations, the tool attempts to force an object's refcount to zero and subsequently access that object via syscalls that share

Tool	LLVM based	Usable in kernel	Public source	Supports refcount	Static/ dynamic
CountDown [9]	Yes	Yes	Yes	Yes	Dynamic
UAFX [10]	Yes	Yes	Yes	No*	Static
Infer [34]	Yes (+ Clang)	Yes	Yes	No	Static
Clang Static Analyzer [33]	Yes	Yes	Yes	No	Static
Syzkaller [35]	Yes	Yes	Yes	No	Dynamic
GCC-fanalyzer [36]	No	No	Yes	No	Static
CRed [37]	Yes	No	No	No	Dynamic
Cppcheck [38]	Yes	No	Yes	No	Static
SVF-derived tools [39]	Yes	No	Yes	No	Static
FreeWill [40]	Yes	No	No	Yes	Dynamic
LinkRID [41]	Yes	Yes	No	Yes	Hybrid
CRCount [42]	Yes	No	No	Yes	Dynamic
UAFChecker [43]	Yes	No	No	No	Static
DCUAF [44]	Yes	Yes	No	No	Static
SinkFinder [45]	Yes	No	No	Yes	Static
Goshawk [46]	Yes	Yes	Yes	No	Static
CID [47]	Yes	Yes	No	Yes	Static

* In theory, UAFX is capable of detecting reference counting issues as well; however, by design, it has been limited to algorithmically filter them out because their tests revealed too many false positives.

Table 1: Complete list of tools that we have considered for this study.

its reference.

Workflow

CountDown's workflow can be broken down into three main phases:

1. **Kernel instrumentation and logging**: CountDown instruments the refcount APIs in the Linux kernel to log every refcount operation using a 4-element tuple:

   $$< S, O, \delta, V >$$

   where a syscall S updates the refcount O by a value δ. V is the final value.

   The instrumented APIs are listed in the following table:

Name	Description
`refcount_set`	set refcount to the given value
`refcount_add`	add the given value to refcount
`refcount_inc`	increase refcount by 1
`refcount_dec`	decrease refcount by 1
`refcount_add_not_zero`	add given value to refcount if refcount is not 0
`refcount_inc_not_zero`	increment if the refcount is not 0
`refcount_dec_not_one`	decrement if the refcount is not 1
`refcount_dec_if_one`	decrement if the refcount is 1
`refcount_dec_and_test`	decrement refcount and check if result is 0
`refcount_sub_and_test`	subtract value from refcount and check it with 0

   To identify the same object across different virtual machines during fuzzing, it uses a checksum of the call stack at the time of the object's initialization through `refcount_set`.

   CountDown creates a RAI map (`refcount-address-ID`) to link the memory addresses holding refcount values with their refcount IDs:

   - when a refcount is initialized, a new entry in the RAI map is created with its address and ID;
   - when a refcount reaches zero, the entry is removed from the map;

- when a refcount is incremented or decremented, CountDown searches the address in the map to find its ID and records the operation.

During fuzzing, to connect refcount operations with syscalls, CountDown records the Syzkaller syscall number (NR) for each operation and considers that one syscall may modify a refcount many times, but it only cares about the final variation (δ). In order to do so, every API listed in table 1 is modified to record every single change. This data is kept in a two-dimension table (one for the syscall number, one for the refcount ID), in which every entry records the current δ and the current V.

2. **Reworking relationships between system calls**: ① CountDown constructs multiple relations among syscalls and refcounts starting from the operations mapped in the 4-tuple format (S, O, δ, V).

   First, it identifies syscalls that effectively change the refcount of any object. ② It projects a set of 4-tuples in a new map, where the key is the refcount ID and the value is a set of syscalls and their changes to the refcount (δ), avoiding relations between syscalls from different programs (which can quickly become unrealiable).

   Based on the shared refcounts, every relation is reworked in a new one defined as `RefcntRelation`. The assumption is that, if two syscalls access the same refcount within one execution, they have a stronger relation with respect to triggering more complicated refcount operations. Each syscall set associated with a specific refcount is then split into a set of syscalls pairs ③. Following that, CountDown creates the syscall relation table, where each unique pair of syscalls has an entry and the value is the number of shared refcounts accessed by both ④.

As explained in the paper, this newly constructed relation is more effective at guiding the fuzzer toward detecting refcount-related UAF bugs. However, the kernel code base is very large and refcount operations are not densely distributed. To solve the problem, the `RefCntRelation` is merged with the Syzkaller's standard `SyzRelation` and handled based on the fuzzing time. At the beginning of fuzzing, CountDown assigns a high priority to the original coverage-based relation to quickly explore more kernel code. As time goes on, it allocates more weight to CountDown's own relation so that the fuzzer can focus on testing complex refcount operations. This can be represented as the following formula:

$$OverallRelation = log_2(SyzRelation) + k * log_2(RefcntRelation)$$

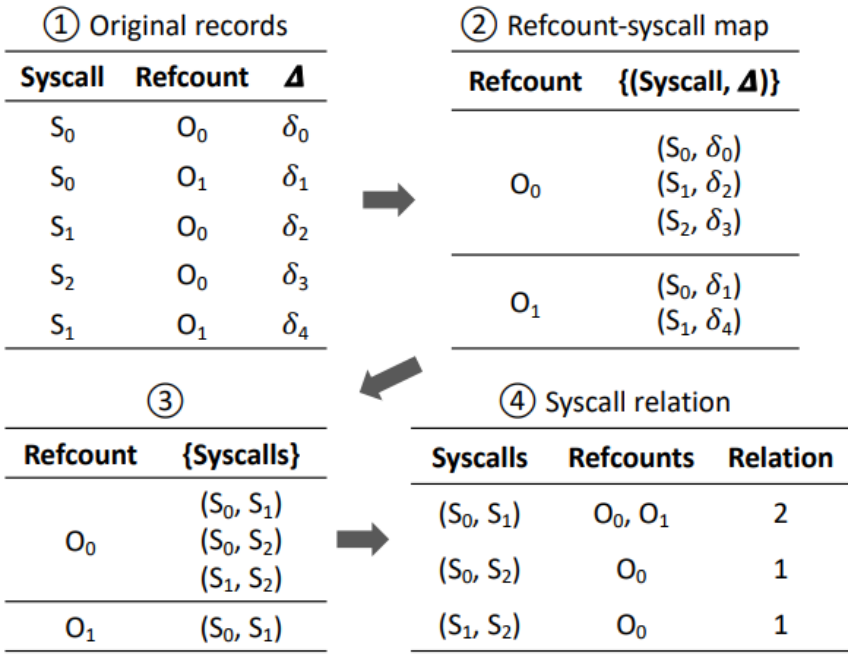


Figure 4: Example of refcount-based syscall relation construction.

k starts with a small value and is incremented every time after testing a fixed number of test cases.

Since fuzzers are non-deterministic, different kernel instances can gather different relations. When a new one is received from one VM, CountDown synchronizes it across all instances (to ensure that the other instances can directly make use of it) by fetching each VM's report, merging them and then distributing the result to all instances.

3. **Guided program mutation**: CountDown uses the learned relationships to generate new programs. It starts by using the mutators from previous fuzzing tools (like Syzkaller) to mutate the program. Having assigned higher weights to refcount-based relations, the mutation will prefer combining syscalls that are strongly related regarding refcount operations.

   Then it creates a new mutator that combines refcount-related syscalls more aggressively. CountDown chooses one target program and runs it in order to collect all accessed refcounts. It will update the refcount-syscall map if the execution triggers new records. To mutate the program, it will randomly select a refcount from the collected set. If a refcount was accessed multiple times, it will have a higher probability to be selected.

   CountDown then searches the map for syscalls that operate on this refcount and selects one for insertion into the program, which is done reusing Syzkaller's logic, preferring

a later position, because the syscalls at the beginning may work on the initialization of resources and refcounts.

In addition to new code coverage, CountDown considers any program that exhibits a new `(syscall, refcount)` pair never seen before to be "interesting" and saves it for future changes. This is done while fuzzing through checking whether the execution triggers new code coverage or a new refcount operation, which was previously missed. If that's the case, it saves the new program into the input queue for further mutations and updates the refcount-syscall map with the new entry.

As a final but important point, a triggered refcount error will not immediately result in a memory error. CountDown needs to trigger more operations to make the kernel release the object and access it through dangling references. It does so by inserting more refcount-related syscalls into the executed program in order to reduce the gap between a refcount bug and a UAF, so that sanitizers like KASAN can capture it.

Limitations

Although CountDown is optimized to detect UAFs with refcounts, it still has some limitations:

- **Generic refcounts**: some refcounts (such as those in the AppArmor security module) are used by nearly all syscalls, creating "noise" and artificial relationships that CountDown must filter out to maintain accuracy.
- **Coverage trade-off**: because it focuses heavily on refcount logic, CountDown may record slightly lower total code coverage (approximately 1.8% – 4.3% less) than Syzkaller, as it ignores paths unrelated to object management.
- **Instrumentation overhead**: although optimized through selective logging (excluding interrupts and asynchronous tasks), the constant monitoring of every refcount operation introduces a higher computational load than traditional fuzzing.

2.4.2 UAFX

UAFX is a static analyzer, presented at NDSS 2025 [10], designed to detect *cross-entry* (non-linear) use-after-free vulnerabilities in the Linux kernel. The term "cross-entry" refers to conditions in which the deallocation and subsequent use of an object occur through different entry points (e.g system calls, callbacks, driver interfaces). In such cases, the link between the deallocation point and the usage point may involve parts distributed across distinct execution paths, making it difficult to detect using analyzers limited to a single function or a single call chain (i.e. where memory allocation, usage, and deallocation occur in a linear pattern). UAFX's approach is based on achieving two main macro-objectives:

- **Escape-Fetch-based Cross-Entry Alias Analysis**
- **Systematic Partial-Order Constraint-based UAF Validation**

We describe them with reference to the following figure:

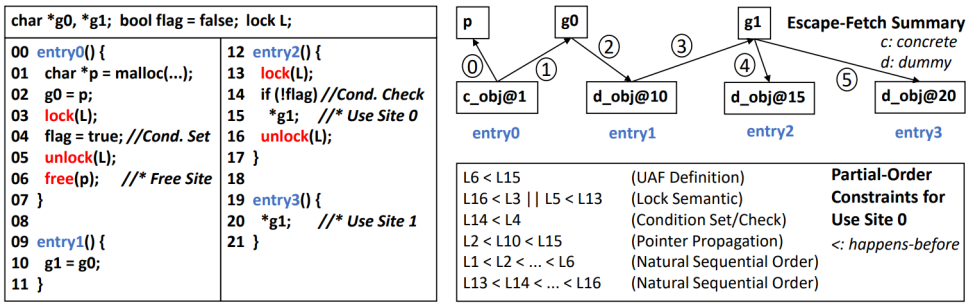


Figure 5: Example of code entry points and how they are handled by UAFX.

Escape-Fetch-based Cross-Entry Alias Analysis

UAFX treats object assignments to pointers as *escapes* and retrievals (from "unknown" areas) as *fetches*. UAFX attempts to construct Escape-Fetch Paths (EFPs) to define aliases that track memory allocation, usage, and deallocation from different access points. This is necessary because, from the perspective of a single function, an object may have an unknown origin (as is commonly the case with an externally initialized pointer, such as `g1` in `entry3`).

Systematic Partial-Order Constraint-based UAF Validation

Once the aliases/paths have been confirmed, UAFX verifies that the conditions for a UAF actually exist. If, for example, there is a check on a flag (`entry0`, `entry2`) that is manipulated by multiple entry functions and used to prevent a UAF on a critical pointer, that path must be discarded.

Workflow

We can break down UAFX's workflow as follows:

1. **Identifying entry functions.** This is done semi-automatically, starting with:
    - *General call analysis*: functions that serve as entry points typically have no callers in the program, so they are candidates.
    - *Software-based heuristics*: the paper uses drivers as an example; these typically have entry points in a `file_operations` structure, which makes them easy to identify.

After these initial automated steps, a manual review is conducted to exclude unnecessary candidates or to add others.

2. **Code analysis and summary for each entry.** The entry functions are analyzed, and all escape/fetch objects, critical regions, and condition sets/checks (such as the flag in the example) are identified. UAFX generates a summary of all possible paths (in Figure 5, the graph in the upper right).

3. **Cross-entry detection of free and use sites with aliases.** UAFX considers all sites where memory regions are freed and used as potential candidates for UAF. In the example, with the path `entry0-entry3`, defined as follows (between the *free* operation on line 6 and the *use* operation on line 20):

$$(\text{c\_obj@1} \longrightarrow \text{g0} \longrightarrow \text{d\_obj@10} \longrightarrow \text{g1} \longrightarrow \text{d\_obj@20})$$

there is a potential UAF.

4. **UAF Validation.** UAFX considers all candidates and evaluates their constraints within the path (order, locks, set/check conditions) to determine whether each one is actually a vulnerability. It also verifies that the free and use sites actually access the same object, taking into account the entire escape-fetch path. For this purpose, SMT solvers can be used (the paper cites z3 [48] as an example).

Thread tracking

Since some UAFs are caused by race conditions, UAFX recognizes two types of thread-related lifecycles:

- **Thread creation**: calls that create child threads (`pthread_create`, `fork`). UAFX retrieves information from these events (e.g., the thread's entry function or parameters)
- **Thread joins** (`pthread_join`, `wait`): events that ensure threads exit before a certain point in the parent thread. UAFX pairs each exit event with its corresponding creation event to track the entire thread lifecycle.

UAFX limitations and extensions

Although UAFX is designed to automatically identify use-after-free vulnerabilities, its analysis deliberately excludes some classes of candidates in order to preserve high precision and limit the number of false positives.

A first limitation concerns the identification of entry functions. As described previously, UAFX automatically considers functions with no callers as potential entry points, but relevant

functions may themselves be invoked by other functions and therefore not satisfy this criterion. UAFX addresses this limitation by allowing additional entry functions to be manually included in the candidate set during the second stage of the identification process.

A more significant limitation concerns reference counting mechanisms. As explicitly discussed by the authors in the *Reference Count Mechanisms* section of the UAFX paper [10], candidates involving reference counting are intentionally excluded from the analysis. Tracking the lifetime of reference-counted objects introduces additional complexity and may produce a large number of false positives. The authors therefore favor higher precision at the cost of reduced recall for this class of lifetime-management errors.

This limitation does not imply that reference-count-related vulnerabilities cannot be addressed through static analysis. Other tools focus specifically on the analysis of reference counting mechanisms and may complement the approach adopted by UAFX. In particular, LinKRID [41], CRCount [42], and FreeWill [40] are discussed in relation to the identification of reference counting errors. Their analyses could potentially be combined with or used to extend UAFX, although doing so would require adapting the respective analyses and reconciling their different representations of object lifetime and reference-management operations.

Such an extension would broaden the class of lifetime errors considered by UAFX, potentially improving its ability to detect use-after-free conditions originating from incorrect reference-count management. However, this would also introduce the precision-recall tradeoff that UAFX deliberately avoids in its original design.

One final "limitation", though not entirely so, is the fact that, in the paper, UAFX appears to have been tested only on drivers. In our case, the tests are performed on linked Linux kernel modules, which, as such, present a more complex challenge than a single-entity context like a driver.

Chapter 3

CVE-2025-21756

3.1 Overview

CVE-2025-21756 [7] is a use-after-free vulnerability identified in the Linux kernel's `af_vsock` [8] subsystem, which is used for communication between hosts and virtual machines. With a specific sequence of operations on vsock sockets, an attacker can trigger a use-after-free condition, resulting in the dereferencing of already-freed memory structures and potential kernel memory corruption. Under certain circumstances, this vulnerability could be exploited to escalate privileges to root [23].

3.2 The vsock subsystem

Vsock is a component of the Linux kernel that manages communication between a virtual machine (*guest*) and the system hosting it (*host*). It was developed to provide a communication method between the two entities with better performances than traditional network interfaces. Unlike classic TCP/IP sockets, vsock removes the overhead of the IP stack and uses an addressing model specific to virtualised environments, based on a pair of identifiers:

- **CID (Context IDentifier)**: uniquely identifies an endpoint (host or VM)
- **Port**: similar to TCP/UDP ports

Some relevant CID values may include:

- `VMADDR_CID_HOST`: represents the host
- `VMADDR_CID_LOCAL`: local communication within the endpoint
- specific CIDs assigned to individual VMs

Communication takes place via two *transport networks*:

- **G2H (guest-to-host)**: they run on the guest and are usually implemented through device drivers. We distinguish between three types:
  - *virtio-transport*: typical of KVM [49]/QEMU [50], it uses virtqueues located within the guest as buffers, but the host also reads from and writes to them.
  - *vmci-transport*: VMWare's [51] proprietary for H2G/G2H.
  - *hyperv-transport*: based on Hyper-V VMBus [52], designed specifically for Microsoft environments.
- **H2G (host-to-guest)**: they run on the host and usually provide device emulation for the guest. There are two types:
  - *vhost-transport*: host-side backend for virtio-transport.
  - *vmci-transport*: VMWare's proprietary for H2G/G2H.

To illustrate communication between the host and the guest, we can use QEMU, an open-source emulator and virtualization program, as an example. When creating a virtual machine, the structure will be as follows:

- *virtio-transport* on the guest: provides the guest-side transport layer used to exchange data with the host through the virtio interface;
- *vhost-transport* on the host: provides in-kernel device emulation to the guest and uses `ioeventfd` and `irqfd` to receive and send events to the guest.

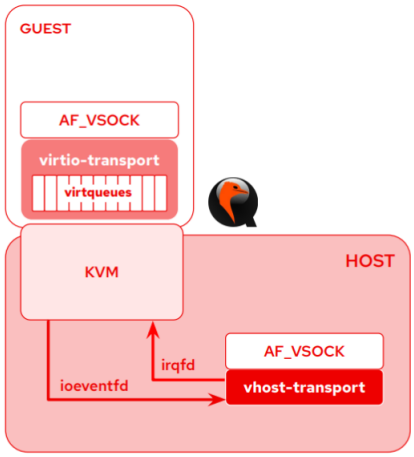


Figure 6: Example of a vsock communication structure.

Both transport layers exchange packets with virtio queues (*virtqueues*).

Virtqueues

These are shared buffer queues in memory that allow the two parties to communicate bidirectionally (even though a single virtqueue is unidirectional) while maintaining low overhead. In **split** mode (classic), virtqueues are implemented using three shared memory areas:

- **Descriptor Area**: contains metadata (like the guest's physical address and length) of allocated buffers that can be used for data exchange. The other rings will reference these buffers using indexes called *descriptors*.
- **Available Ring**: a list of descriptors that the guest driver has added and which are ready to be consumed by the host device.
- **Used Ring**: a list of descriptors that the device has finished processing and is returning to the guest driver.

The **packed** virtqueue mode, on the other hand, uses a more modern and compact layout, merging all the three rings into a single one thereby improving the performance of the processor cache. The single ring is managed similarly to the previously mentioned *descriptor area*: it contains a list of buffer descriptors that are no longer referenced by index, but rather through an opaque value stored within the descriptor itself. The descriptor flags now account for the two missing rings: one flag indicates whether the descriptor is available for use, while another indicates whether the buffer is currently in use [53].

The virtqueue backend (which actually manages the queue on the host) can be **QEMU**, with full emulation, as in the case under consideration, or **vhost**: the backend runs within the host kernel (vhost-net/vhost-blk), allowing the driver to communicate directly with the kernel without going through QEMU's user space, thereby drastically reducing latency.

Internal architecture of vsock

From an implementation perspective, each vsock socket is represented in the kernel with the structure `vsock_sock` [54], which extends the generic one `sock` [28] associated to sockets. Lifetime of these objects is governed by the reference counting mechanism, and they can be associated with one of the following lists:

- *unbound*: contains sockets that are not associated with a port,
- *bound*: contains sockets associated with a port,
- *connected*: contains sockets that are both bound and connected.

When consistency in the management of these elements is lacking, there is a risk of encountering the problems mentioned above, exposing the system to risks and attacks.

The life cycle of a vsock socket

Typically, a vsock socket goes through the following stages:

1. **Creation** (`socket()`): memory is allocated for the data structure, which is then added to the *unbound* list.
2. **Binding** (`bind()` or autobind): the socket is associated with a port and is consequently added to the *bound* list.
3. **Connection** (`connect()` / `accept()`): the actual communication channel is established.
4. **Release** (`close()`): when the socket is no longer needed, it is removed from the internal structures, its reference count is decremented and, if this reaches zero, its memory is deallocated.

3.3 Test case presented by the developers

Let's start with a test case [55] provided by the maintainers of the vsock module for the Linux kernel, who, after releasing the patch, made it available to allow users to check whether their system is affected by the issue by triggering a use-after-free and crashing the system.
The test case in question does not lead to a privilege escalation, but causes the kernel to crash on vulnerable systems. The complete code is reported in Listing 3.1.

Step-by-step analysis

Let's look at every relevant step in the test case to later understand what is going wrong in the reference counting mechanism associated with a vsock socket. Every kernel-side functionality involved will be described in detail in section 3.4. Error handling code is omitted when not necessary.

```
14      s = vsock_bind(VMADDR_CID_LOCAL, VMADDR_PORT_ANY, SOCK_SEQPACKET);
```

The Proof of Concept (POC) starts by creating a local (`VMADDR_CID_LOCAL`) socket bound to the first available port (`VMADDR_PORT_ANY`).

```
26      getsockname(s, (struct sockaddr *)&addr, &alen)
27
28      for (i = 0; i < MAX_PORT_RETRIES; ++i) {
29          sockets[i] =
30              vsock_bind(VMADDR_CID_ANY, ++addr.svm_port, SOCK_SEQPACKET);
31      }
32
33      close(s);
```

Starting with the port immediately following the one assigned by the kernel to the local socket, the code arbitrarily binds 24 ports in order to occupy them. This value corresponds to the maximum number of attempts that vsock makes to find an available port when a port is assigned automatically [56]. The goal is to cause all automatic attempts to fail by manually occupying all the ports. The first socket is then closed – its only purpose was to automatically obtain the first available port from the kernel.

```
35      s = socket(AF_VSOCK, SOCK_SEQPACKET, 0);
36
37      if (!connect(s, (struct sockaddr *)&addr, alen)) {
38          fprintf(stderr, "Unexpected connect() #1 success\n");
39          exit(EXIT_FAILURE);
40      }
```

In this step, the "victim" socket for the exploit is created with no port, no peer, no bind, and no transport. Note that, when the socket is created, its reference counter is 2. This is because the first reference belongs to the `struct socket`, and the second is the reference associated with adding the socket to the unbound list, which is done by `__vsock_create()`.

Then, we want the `connect` to fail in a specific way. With this connection attempt, the kernel prepares the socket, selects the transport and attempts to assign it a local port. The 24 ports we set up earlier are used to steer the process along this error path so that the transport is set but the socket remains in the unbound list.

```
47      addr.svm_cid = VMADDR_CID_NONEXISTING;
48      if (!connect(s, (struct sockaddr *)&addr, alen)) {
49          fprintf(stderr, "Unexpected connect() #2 success\n");
50          exit(EXIT_FAILURE);
51      }
52      /* connect() #2 failed: transport unset, sk ref dropped? */
```

The CID is then set to a non-existent type to force the next connection attempt to fail; this one proceeds just like the previous one, and we again expect it to fail, but for a different reason. This causes the trasport to be unset and, at the same time, the reference count to be decreased to 1 even if the object is still existing.

```
54      addr.svm_cid = VMADDR_CID_LOCAL;
55      addr.svm_port = VMADDR_PORT_ANY;
56
57      /* Vulnerable system may crash now. */
58      bind(s, (struct sockaddr *)&addr, alen);
59
60      close(s);
```

```
61      while (i--)
62          close(sockets[i]);
63
64      control_writeln("DONE");
```

In the `bind()` statement, the reference counter is further decremented to 0 when the socket is removed from the list of unbound sockets, causing the resource to be released. When the kernel-side `__vsock_bind` function continues execution and adds the socket to the bound list, the UAF is triggered.

This final step demonstrates how the vulnerability was triggered, showing that, on a vulnerable system, attempting to bind a socket to an invalid object causes a crash.

3.4 Cause and functions involved

In order to understand exactly why the bug occurs, we can start by looking at the patch [57] the kernel developers applied to fix it. Before the patch (kernel versions until 6.6.78 [54]), the vulnerable code in `af_vsock` looked like this:

```
1       void vsock_remove_sock(struct vsock_sock *vsk)
2       {
3           vsock_remove_bound(vsk);
4           vsock_remove_connected(vsk);
5       }
```

while in the version after the patch (6.6.79) it looked like this:

```
1       void vsock_remove_sock(struct vsock_sock *vsk)
2       {
3               /* Transport reassignment must not remove the binding. */
4               if (sock_flag(sk_vsock(vsk), SOCK_DEAD))
5                       vsock_remove_bound(vsk);
6
7               vsock_remove_connected(vsk);
8       }
```

The maintainers added a simple `if` that checks if the socket is really "dead" before removing the binding. We can better understand how it works by looking backward, starting with the call to the `vsock_remove_bound` [54] function.

Before examining the function itself, we need to explain how the **bound tables** approach works. There are two global (as all sockets on the system are placed within them) tables, one for **bound sockets** (sockets that have specified an address that they are responsible for) and

one for **connected sockets** (sockets that have established a connection with another socket). The bound table contains an extra entry for a list of **unbound sockets**. These tables are managed as *intrusive lists* and are contained directly within the socket structs.

In the test case, the `SOCK_SEQPACKET` socket created after the port exhaustion loop is initially placed (by the kernel) in the unbound list because no local address has yet been assigned to it. When `connect()` to a local address, the kernel implicitly attempts to bind the socket. However, this operation fails because the previously allocated consecutive ports are unavailable. Consequently, the socket remains in the unbound list, despite having a transport layer assigned.

The second `connect()` targets a non-existent address, causing the kernel to force a new transport selection. During this reassignment, the previous transport is released and the socket is removed from the bound list through the normal bound-socket removal path, but the socket was never actually moved from the unbound list to one of the regular bound-table buckets. The vulnerable kernel code therefore treats an unbound socket as if it were normally bound and drops a reference that should have been preserved. The subsequent `bind()` then operates on a socket whose reference count may already be incorrect, exposing the use-after-free condition.

Now that we've covered how bound tables work, we can examine the contents of `vsock_remove_bound`. Since the original function calls a wrapper that contains mutual exclusion instructions and the entire function that contains the actual logic, for simplicity we'll focus directly on the latter:

```
1       static void __vsock_remove_bound(struct vsock_sock *vsk)
2       {
3               list_del_init(&vsk->bound_table);
4               sock_put(&vsk->sk);
5       }
```

The `list_del_init` statement removes the socket from the bound table, while the `sock_put` function contains the statement responsible for decrementing the reference counter for the current socket and, if necessary, destroying it if the decremented reference was the last one:

```
1       static inline void sock_put(struct sock *sk)
2       {
3               if (refcount_dec_and_test(&sk->sk_refcnt))
4                       sk_free(sk);
5       }
```

This means that, if the socket was not actually unbound, its reference counter was still decremented and the memory was freed, even though a valid reference to it still existed. Now that we understand why this vulnerability exists, let's analyze the options available to us to identify the vulnerability with the tools we've considered and how they interact with the affected modules in the kernel version under review.

Listing 3.1: Test case presented by the developers.

```c
#define MAX_PORT_RETRIES        24
#define VMADDR_CID_NONEXISTING  42

/* Test attempts to trigger a transport release for an unbound
 * socket. This can lead to a reference count mishandling.
 */
static void test_seqpacket_transport_uaf_client(
    const struct test_opts *opts
) {
    int sockets[MAX_PORT_RETRIES];
    struct sockaddr_vm addr;
    int s, i, alen;

    s = vsock_bind(
        VMADDR_CID_LOCAL,
        VMADDR_PORT_ANY,
        SOCK_SEQPACKET
    );

    alen = sizeof(addr);
    if (getsockname(s, (struct sockaddr *)&addr, &alen)) {
        perror("getsockname");
        exit(EXIT_FAILURE);
    }

    for (i = 0; i < MAX_PORT_RETRIES; ++i) {
        sockets[i] = vsock_bind(
            VMADDR_CID_ANY,
            ++addr.svm_port,
            SOCK_SEQPACKET
        );
    }

    close(s);
    s = socket(AF_VSOCK, SOCK_SEQPACKET, 0);
    if (s < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    if (!connect(s, (struct sockaddr *)&addr, alen)) {
        fprintf(stderr, "Unexpected connect() #1 success\n");
        exit(EXIT_FAILURE);
    }
    /* connect() #1 failed: transport set, sk in unbound list. */

    addr.svm_cid = VMADDR_CID_NONEXISTING;
```

```c
    if (!connect(s, (struct sockaddr *)&addr, alen)) {
        fprintf(stderr, "Unexpected connect() #2 success\n");
        exit(EXIT_FAILURE);
    }
    /* connect() #2 failed: transport unset, sk ref dropped? */

    addr.svm_cid = VMADDR_CID_LOCAL;
    addr.svm_port = VMADDR_PORT_ANY;

    /* Vulnerable system may crash now. */
    bind(s, (struct sockaddr *)&addr, alen);

    close(s);
    while (i--)
        close(sockets[i]);

    control_writeln("DONE");
}
```

Chapter 4

CountDown testing: dynamic analysis

The first test we conducted aimed to detect the UAF described in the previous chapter using CountDown [9]. The test was conducted in two phases: first, we tried running CountDown in unguided mode (to simulate a realistic situation where the cause of the bug is unknown) and then we ran it with a starting corpus to direct the fuzzer toward the vulnerability under examination.

4.1 Setup

We started by configuring KVM (following the official instructions [58]) on a Linux machine. To do this, we used a system running Fedora 41. Next, we retrieved the CountDown source code from the official repository [59] and installed the necessary dependencies.

We then downloaded the Linux kernel source code from the official repository. We used version 6.6, which was impacted by the vulnerability. The reason for this choice was that we wanted to use a version on which CountDown had been tested, as specified in the paper, to avoid issues that might arise from using an untested version. To verify that the kernel version in question was indeed affected by the bug under investigation, we ran the developers' test case on a virtual machine created with QEMU [50], ensuring that the system crashed upon reaching the last `bind()` instruction.

Once the kernel source was downloaded, we applied the CountDown-specific patch to the kernel source tree. This patch instruments the kernel's reference counting operations (`kref_get` and `kref_put`) to allow CountDown to monitor refcount changes at runtime and detect anomalies (in our case, increments on a zero refcount, which are indicative of use-after-free conditions).

```
1 git apply ../kernel_compile/kernel.patch
```

We then configured the kernel using the configuration file provided by CountDown, which enables the options required by the fuzzer's core (Syzkaller). Some examples that can be found

in Syzkaller's documentation [60] are `CONFIG_KCOV` for coverage collection and `CONFIG_KASAN` for memory error detection.

```
cp ../kernel_compile/.config ./
make olddefconfig
```

During the configuration step, we were prompted for a small number of options not present in the provided configuration file. Specifically, we were asked to choose:

- a stack variable initialization policy, for which we selected `INIT_STACK_NONE` [61] (no automatic initialization) to avoid masking memory bugs;
- about enabling heap zeroing on allocation (`INIT_ON_ALLOC_DEFAULT_ON`) to improve determinism in memory behavior [62];
- about disabling heap zeroing on free (`INIT_ON_FREE_DEFAULT_ON`) [63] to preserve the stale memory content that KASAN relies on to detect use-after-free accesses.

We then compiled the kernel:

```
make CC="gcc -std=gnu11" HOSTCC="gcc -std=gnu11" \
     KBUILD_CFLAGS="-std=gnu11" KBUILD_AFLAGS="-std=gnu11" \
     -j$(nproc)
```

The `-std=gnu11` flag was necessary due to a compatibility issue between the kernel 6.6 codebase and GCC 15, the version available on our Fedora system: GCC 15 defaults to the C23 standard, in which `bool` and `false` are reserved keywords, causing compilation errors in kernel headers that redefine them.

After successfully compiling the kernel, we proceeded to build a Debian-based disk image using the script provided in the CountDown repository:

```
cd ../image
chmod +x ./create-image.sh
./create-image.sh
```

This script uses `debootstrap` [64] (a tool used to install a base Debian system into a target directory on an already running Linux system) to create a minimal Debian (Bookworm) root filesystem, configures SSH access with a generated key pair, and produces a raw disk image suitable for use with QEMU.

We then compiled CountDown itself:

```
cd ../countdown_fuzzer
make
```

After that, we configured the fuzzer by editing the `cd_cfg` file, specifying the paths to the compiled kernel, disk image, and SSH key, as well as the virtual machine parameters: 6 parallel VM instances, 2 virtual CPUs each, and 2048 MB of RAM per instance, for a total of 12 vCPUs and 12 GB of RAM allocated to the fuzzer. At this point, the workflow splits into two different approaches, described below.

4.2 Tests without starting corpus

For the first test, we ran CountDown as configured above, without any further changes:

```
./bin/syz-manager -config ./cd_cfg
```

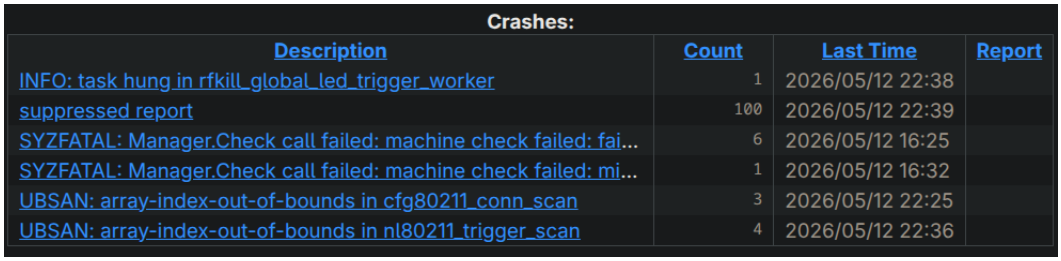
Upon startup, `syz-manager` loads the corpus, starts serving an HTTP dashboard for real-time monitoring and boots the configured virtual machine instances. Once the guest VMs are running and connected, the fuzzer begins executing test programs and collecting coverage and signal data, as shown in the following example output:

```
2026/05/12 16:33:34 machine check:
2026/05/12 16:33:34 syscalls              : 3699/4372
2026/05/12 16:33:34 code coverage          : enabled
2026/05/12 16:33:34 comparison tracing     : enabled
2026/05/12 16:33:34 fault injection        : enabled
2026/05/12 16:33:34 leak checking          : CONFIG_DEBUG_KMEMLEAK is not enabled
```

The `machine check` output confirms that the key fuzzing features are active: code coverage collection via `CONFIG_KCOV`, comparison tracing and fault injection [65]. The absence of `CONFIG_DEBUG_KMEMLEAK` [66] is not critical for our purposes, as KASAN [67] is sufficient to detect use-after-free conditions.

In this first test, we ran CountDown without providing any seed corpus, allowing the fuzzer to operate in its default mode: generating and mutating syscall sequences autonomously, guided solely by coverage feedback. This approach reflects the typical black-box fuzzing scenario, where no prior knowledge of the vulnerability is assumed. We decided to try it this way to see if the bug could be detected without manual intervention. Despite running the fuzzer for several days, CountDown did not produce any crash report attributable to CVE-2025-21756, as we can see in figure 7. During the tests, the `crashes/` directory in the workdir remained either empty or contained only unrelated kernel warnings. The kref history files (`kref-relation-cross-history` and `kref-refcnt-change-cross-history`) remained empty, indicating that the refcount-guided instrumentation did not observe any anomalous reference counting behavior along the explored paths.

This result is consistent with the nature of the vulnerability: the syscall sequence required to trigger CVE-2025-21756 is highly specific, involving a precise ordering of `socket`, `bind`,



Crashes:

Description	Count	Last Time	Report
INFO: task hung in rfkill_global_led_trigger_worker	1	2026/05/12 22:38	
suppressed report	100	2026/05/12 22:39	
SYZFATAL: Manager.Check call failed: machine check failed: fai...	6	2026/05/12 16:25	
SYZFATAL: Manager.Check call failed: machine check failed: mi...	1	2026/05/12 16:32	
UBSAN: array-index-out-of-bounds in cfg80211_conn_scan	3	2026/05/12 22:25	
UBSAN: array-index-out-of-bounds in nl80211_trigger_scan	4	2026/05/12 22:36	

Figure 7: Report of the first unguided test with CountDown.

and `connect` calls with particular CID values. Without guidance, the probability of a random fuzzer generating this exact sequence is extremely low, especially considering our performance capabilities.

4.3 Tests with starting corpus

To improve the chances of triggering the vulnerability, we adopted a guided fuzzing approach by providing CountDown with a seed corpus, a collection of syscall programs that the fuzzer uses as starting points for mutation: rather than generating programs from scratch, the fuzzer mutates and recombines existing seeds, exploring the neighborhood of the provided sequences in the program space. This mechanism allowed us to encode prior knowledge about a vulnerability directly into the fuzzing process, reducing the search space.

The seed we provided encodes the syscall sequence known to trigger CVE-2025-21756, based on the analysis described in chapter 3 and on the test case published by the kernel maintainers alongside the patch [57]. We translated this sequence into Syzkaller's typed syscall format, which uses resource-aware types (e.g. `socket$vsock_stream`) rather than raw syscall numbers, allowing the fuzzer to mutate arguments in a semantically meaningful way:

```
r0 = socket$vsock_stream(0x28, 0x1, 0x0)
bind$vsock_stream(r0, &(0x7f0000000000)={0x28, 0x0, 0x1, 0x0}, 0x10)
r1 = socket$vsock_stream(0x28, 0x1, 0x0)
connect$vsock_stream(r1, &(0x7f0000000300)={0x28, 0x0, 0x1, 0x4}, 0x10)
connect$vsock_stream(r1, &(0x7f0000000400)={0x28, 0x0, 0x2a, 0x5}, 0x10)
bind$vsock_stream(r1, &(0x7f0000000500)={0x28, 0x0, 0x1, 0xffffffff}, 0x10)
```

This sequence reflects the vulnerability trigger explained in section 3.3: the first `connect` to `VMADDR_CID_LOCAL` sets the transport and places the socket in the unbound list; the second `connect` to a non-existing CID (42) unsets the transport and incorrectly decrements the reference counter; the subsequent `bind` operates on the freed object, triggering the UAF.

The seed was imported into the corpus using the `syz-db` tool provided by syzkaller:

```
./bin/syz-db pack /tmp/vsock_seed /tmp/vsock_seed.db
./bin/syz-db merge workdir/corpus /tmp/vsock_seed.db
```

We also explicitly restricted the fuzzer to vsock-related syscalls via the `enable_syscalls` field in the configuration file, so that the mutation process would remain focused on the relevant code paths rather than dispersing coverage across unrelated kernel subsystems:

```
"enable_syscalls": [
    "socket$vsock_stream",
    "bind$vsock_stream",
    "connect$vsock_stream",
    "socket$vsock_dgram",
    "bind$vsock_dgram",
    "connect$vsock_dgram",
    "accept4$vsock_stream",
    "getsockname",
    "close"
]
```

However, this does not guarantee that CountDown will follow the same steps as the test case, but only that it will use the specified system calls as a starting point for the scan which, as can be seen in the example in the figure 8, were used to construct various paths.

The reason for this is that Syzkaller, which underlies CountDown, modifies every input involved in the instruction sequence, in our case, the syscall parameters. This, in particular, prevents us from exactly replicating the 24-bind phase to occupy the initial ports, for two reasons:

1. we can configure the corpus to contain 24 bind instructions. As the port parameter, however, we would need a value that is one higher than the one automatically configured by the first bind, which we do not have in this case. The function's return value is, in fact, the integer corresponding to the operation's result (or any error code), while the port can only be derived as a side effect, something we are not permitted to do;

2. even if we could potentially perform the operation described in point 1, we would be going against the very logic of CountDown (and Syzkaller), because the syscall input would be altered after the first execution anyway, rendering the acquisition of the first port pointless.

These conditions are a crucial factor in understanding why an unguided tool like CountDown might struggle to detect a bug like this.

Despite this, one important aspect worth mentioning regarding how the corpus works is the incremental construction of the set of followed paths. As Syzkaller (and tools derived from it, such as CountDown) follows the paths, it saves them as a starting point so that, on the next

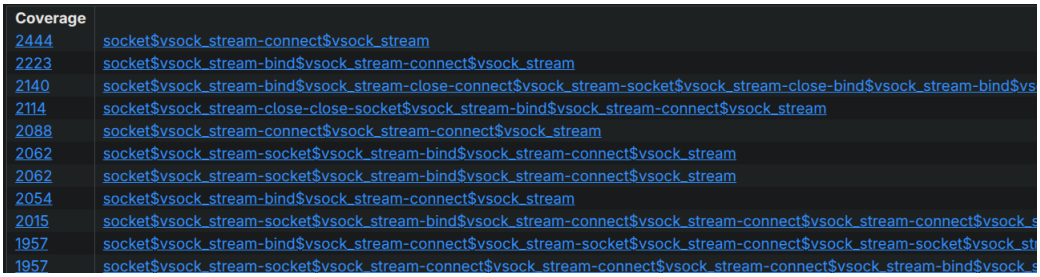
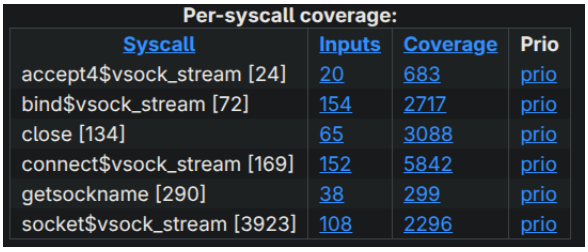


Figure 8: CountDown coverage during one of the tests.

run, it can resume from where it left off. This has allowed us to divide the test execution into multiple steps without losing what has already been scanned and, most importantly, without repeating the same paths multiple times, thereby saving time.

We can further illustrate how it works with the figure 9, which shows the per-system-call coverage reported by CountDown. For each enabled syscall, the *Inputs* column indicates the number of inputs stored in the corpus that exercise that syscall, while the *Coverage* column reports the amount of kernel code coverage reached through those inputs. As the fuzzing process progresses and new executions reach previously unexplored code, the corresponding inputs are added to the corpus.



Per-syscall coverage:

Syscall	Inputs	Coverage	Prio
accept4$vsock_stream [24]	20	683	prio
bind$vsock_stream [72]	154	2717	prio
close [134]	65	3088	prio
connect$vsock_stream [169]	152	5842	prio
getsockname [290]	38	299	prio
socket$vsock_stream [3923]	108	2296	prio

Figure 9: CountDown coverage table during one of the tests.

Figure 10 shows an example of one of the executions launched during our tests. In this particular run, the corpus contains 475 retained inputs and the fuzzer reached an overall coverage value of 9935 while exercising six enabled system calls. During the campaign, more than 1.7 million test executions were performed and 844 new inputs were identified, while no crashes or distinct crash types were detected. Among the reported statistics, the most relevant are:

- `uptime`: the elapsed time since the current CountDown instance was started;
- `fuzzing`: the cumulative amount of time spent performing fuzzing operations;
- `corpus`: the number of inputs currently retained in the corpus;
- `signal`: the amount of execution feedback collected and used to identify interesting inputs;

- `coverage`: the amount of kernel code reached by the generated executions;
- `syscalls`: the number of system calls enabled for the fuzzing campaign;
- `new inputs`: the number of newly discovered inputs considered interesting enough to be added during the execution;
- `exec total`: the total number of test-case executions performed;
- `crashes` and `crash types`: respectively, the number of detected crashes and the number of distinct crash classes;
- `vm restarts`: the number of times the virtual machines used by the fuzzing environment were restarted.

Despite running the fuzzer for several additional days with this guided configuration, CountDown didn't produce a crash report attributable to the CVE-2025-21756. The kref history files also remained empty, confirming that the refcount instrumentation did not intercept any anomalous behavior. This outcome suggests that, even with our targeted seed, the fuzzer was unable to find the precise conditions required to trigger the vulnerability (at least in our time window and with our performance limits), a finding that itself constitutes a meaningful result, as it highlights a limitation of coverage-guided fuzzing when applied to vulnerabilities that depend on subtle interactions between specific syscall arguments and kernel state.

Stats:	
revision	4fa91022
config	config
uptime	3h57m2s
fuzzing	15h41m0s
corpus	475
triage queue	0
signal	13013
coverage	9935
syscalls	6
buffer too small	0 (0/hour)
crash types	0 (0/hour)
crashes	0 (0/hour)
exec candidate	4 (1/hour)
exec collide	1134061 (79/sec)
exec fuzz	461528 (32/sec)
exec gen	4715 (19/min)
exec hints	26616 (112/min)
exec minimize	16962 (71/min)
exec seeds	1009 (256/hour)
exec smash	105260 (445/min)
exec total	1776184 (125/sec)
exec triage	26029 (110/min)
executor restarts	64 (16/hour)
max signal	33853 (143/min)
new inputs	844 (214/hour)
rotated inputs	243 (61/hour)
suppressed	0 (0/hour)
vm restarts	16 (4/hour)

Figure 10: CountDown running table during one of the tests.

Chapter 5

UAFX testing: static analysis

The second test we conducted was the one with UAFX. Although the paper [10] states that UAFs candidates associated to refcount operations are not considered, we decided to test the tool as-is to see if it would still be able to detect the vulnerability, and then to modify it to remove the limitations imposed by the researchers and observe any changes in the results.

An important point is that, in the UAFX paper, all tests were conducted on drivers, which are much smaller contexts than the entire Linux kernel. Therefore, one of our goals was to understand how to run UAFX on a set of modules extracted from the kernel that do not operate in isolation.

5.1 Setup

We started by downloading the kernel source code for version 6.6.75 from the official repository, a vulnerable version that is suitable for CVE-2025-21756 and officially supported by UAFX. Similar to what we did with CountDown, we verified that the test case caused a crash by using a virtual machine created with QEMU running the kernel version in question.

To set up the environment, we installed the necessary dependencies (the most important of which include Python and LLVM version 14), `gclang`, a wrapper for the Clang compiler used to compile the source code of the relevant modules in C and generate LLVM bitcode files, and `gllvm`, a Go porting of LLVM which relies on it to extract the bitcode from the kernel source files.

UAFX also requires z3 [48], an SMT solver [68] (a type of tool that checks whether a logical formula is satisfiable based on mathematical rules). It acts as a logic engine and, in our case, is used to check whether a certain path is feasible during static analysis.

Before moving on, let's introduce some terms used in the explanation:

- **bitcode** (`.bc`): represents the target code in LLVM Intermediate Representation (IR), the level at which UAFX performs its static analysis passes. This intermediate form is

essential because it allows the tool to accurately analyze pointers and perform escape-fetch analysis, which require a code structure with more semantic information than the final binary to map the relationships between pointers even when they span multiple kernel entry functions.

- **object files** (`.o`): the result of partially compiling the source code into machine code. The `.o` files are generated alongside the `.bc` files to ensure that the build environment is consistent. In our case, while the security analysis is performed on the bitcode, the original object files serve to confirm that the code is "compilable" and ready to be converted into a version that the tool can analyze.

After the initial steps, we considered two possible alternatives:

1. compiling with `gllvm` [69]/`wllvm` [70]: these wrappers produce standard binaries and retain a reference to the bitcode, from which a single `.bc` file is then extracted;
2. building the kernel with pure Clang/LLVM + a custom pipeline for merging `.o.bc` files.

Since UAFX requires a single `.bc` file by default, we decided to go with the first option. `gllvm` and `wllvm` act as compiler wrappers; they generate the `.bc` files for the objects, and then `extract-bc` or `get-bc` link them into a single module, on which we can run the tool directly.

We configured some bash environment variables to ensure that LLVM and Clang use version 14:

```
export PATH=/usr/lib/llvm-14/bin:$PATH:$(go env GOPATH)/bin
export LLVM_COMPILER=clang
export LLVM_COMPILER_PATH=/usr/lib/llvm-14/bin
export LLVM_CC_NAME=clang-14
export LLVM_CXX_NAME=clang++-14
export LLVM_LINK_NAME=llvm-link
export LLVM_AR_NAME=llvm-ar-14
```

We also needed to setup the kernel source code in order to build it and extract the bitcode:

```
make mrproper
make LLVM=-14 defconfig
make LLVM=-14 menuconfig
```

These commands allow us to define which modules we're going to launch UAFX on, making the compiled kernel a valid target. Specifically:

- `make mrproper` performs a complete cleanup of the kernel source tree, removing not only the object files and binaries generated by previous builds (as `make clean` would

do), but also the `.config` configuration file and all automatically generated files, such as the header files produced during the build process. This ensured that we started with a completely clean source tree, eliminating any residual files that could affect the subsequent compilation. This has been particularly useful for us when running different tests with different configurations, so that we could always start from a neutral baseline.

- `make LLVM=-14 defconfig` generates a default kernel configuration for the current architecture, using Clang/LLVM version 14 as the compilation toolchain instead of GCC [36]. The configuration generated by `defconfig` includes a minimal set of default options, suitable for most test environments, and served as a starting point for our manual adjustments.

- `make LLVM=-14 menuconfig` opens an interactive menu-based interface that allows us to modify the configuration generated in the previous step. In our context, it was used to explicitly enable all components of the vsock subsystem (`VSOCK`, `VSOCK_LOOPBACK`, `VSOCK_DIAG`, `VMW_VSOCK_VIRTIO_TRANSPORT`, and `VMW_VSOCK_VIRTIO_TRANSPORT_COMMON`), so that the compiled kernel was a suitable target for the analysis performed with UAFX.

Next, we have the final configuration steps and the launch of the actual build:

```
1       make LLVM=-14 olddefconfig
2       make -j"$(nproc)" LLVM=-14 prepare modules_prepare scripts
```

With `make LLVM=-14 olddefconfig`, we reset the `.config` file to the default options for all configuration entries that were not explicitly set via `menuconfig`, because after any manual changes to the configuration, there may be unresolved symbols or unset values that could cause problems during compilation.

The following `make` command performs a series of preparatory steps that are essential for compiling external modules (in our case, UAFX). Below is a detailed explanation of the parameters:

- `prepare` generates the headers and internal kernel data structures that modules need to know about at compile time, including `autoconf.h` (an auto-generated header file created during the Linux kernel build process which translates configuration options from the `.config` file into C preprocessor `#define` macros used to compile kernel modules) and the symbol tables;

- `modules_prepare` extends the preparation process to include everything needed to compile loadable modules (`.ko`), specifically the dependency files and version data for the symbols exported by the kernel (`Module.symvers`);

- `scripts` compiles the auxiliary build tools located in `scripts/`, such as `recordmcount` and the various helper programs used by the kernel build system;

- `-j"$(nproc)"` parallelizes the work across all available logical cores, speeding up the compilation steps.

Once the environment is set up, we start with the actual compilation:

```
make -j"$(nproc)" LLVM=-14 \
    CC=gclang HOSTCC=clang-14 HOSTCXX=clang++-14 \
    LD=ld.lld-14 AR=llvm-ar-14 NM=llvm-nm-14 \
    OBJCOPY=llvm-objcopy-14 \
    OBJDUMP=llvm-objdump-14 \
    STRIP=llvm-strip-14 \
    net/vmw_vsock/vsock.o \
    net/vmw_vsock/vmw_vsock_virtio_transport.o \
    net/vmw_vsock/vmw_vsock_virtio_transport_common.o \
    net/vmw_vsock/vsock_loopback.o \
    net/vmw_vsock/vsock_diag.o \
    net/socket.o
```

This command initiates selective compilation of only the object modules required for the analysis, explicitly specifying the entire LLVM/Clang toolchain to be used. Just as we did with the previous one, let's take a look at its structure:

- `CC=gclang` replaces the C compiler with gclang, which, as we mentioned earlier, not only produces the standard object files but also generates the corresponding LLVM bitcode in parallel, which will be used by UAFX;
- `HOSTCC=clang-14` and `HOSTCXX=clang++-14` specify the compiler to use for build tools run on the host (such as the tools in `scripts/`), which is different from the target compiler;
- `LD`, `AR`, `NM`, `OBJCOPY`, `OBJDUMP`, and `STRIP` replace the corresponding `binutils` tools (used in the compilation process) with their LLVM counterparts.

After the `STRIP` parameter, we have a set of targets that correspond to the vsock subsystem modules selected during the `menuconfig` phase, plus `net/socket.o`, which implements the generic socket interface at the kernel level and is required to resolve symbol dependencies. The result is a set of `.o` files, from which we extract the bitcode:

```
get-bc -o
    ../uafx/_bc/vsock.bc
    net/vmw_vsock/vsock.o
get-bc -o
    ../uafx/_bc/vmw_vsock_virtio_transport.bc
```

```
    net/vmw_vsock/vmw_vsock_virtio_transport.o
get-bc -o
    ../uafx/_bc/vmw_vsock_virtio_transport_common.bc
    net/vmw_vsock/vmw_vsock_virtio_transport_common.o
get-bc -o
    ../uafx/_bc/vsock_loopback.bc
    net/vmw_vsock/vsock_loopback.o
get-bc -o
    ../uafx/_bc/vsock_diag.bc
    net/vmw_vsock/vsock_diag.o
get-bc -o
    ../uafx/_bc/socket.bc
    net/socket.o
```

Here we use `get-bc`, a utility from the wllvm toolkit (on which gllvm is based), which retrieves the LLVM bitcode previously embedded by gclang within each `.o` object file and exports it to standalone `.bc` files. As mentioned, LLVM bitcode is an intermediate representation of the source code, independent of the target architecture, that preserves high-level semantic information that would otherwise be lost in traditional compilation to machine code. The elements of interest to us, for use with UAFX, are the types, the call graph, and the relationships between instructions (which are extremely important for escape-fetch-based cross-entry alias analysis, specifically in the reconstruction of object paths). The `socket.bc` file is included to allow UAFX to resolve the symbols in the kernel's generic socket layer that the vsock modules reference.

As the final step in the general configuration, we set up the list of functions to be considered as entry points by UAFX, a partial list of which is provided below:

```
vsock_create
vsock_bind
vsock_remove_sock
...
__sys_socket
__sys_bind
__sys_connect
...
```

It is important to note that some of the functions we have included are defined within the Linux kernel as **syscalls** (system calls) [71]. They constitute the interface through which user-space programs request services from the kernel (e.g. creating a socket, establishing a connection) passing through syscall-specific macros, which generate the architecture-dependent entry points and the intermediate wrappers required to dispatch a request to its actual implementation. For example, a user-space operation like `connect()` traces back to kernel functions

such as `__sys_connect()`, before being dispatched to the protocol-specific implementation through the socket operation interface.

This presented another potential obstacle because the UAFX paper does not mention any support for functions of this type, nor does it explicitly rule them out. Since the objects we are considering necessarily go through these functions as well, we had to add them to the set of entry functions for UAFX.
At this point, we had two options:

1. include them directly as separate modules (even though the paths between modules would remain incomplete);

2. link the modules and run UAFX on a cumulative bitcode.

The first option involved running UAFX on the modules separately. In this case, only UAFs related to issues within individual modules would have been detected (and thus much less likely to have gone undetected than a UAF affecting multiple modules). With this approach, the configuration ends here.

The second method, on the other hand, requires a few additional configuration steps. As mentioned earlier, the goal was to combine all the modules in question into a single cumulative module so that it could be fed to UAFX all at once, taking into account all the functions of all the modules.
As a first step in this process, we had to disassemble some of the modules:

```
llvm-dis-14
    vsock_loopback.bc
    -o /tmp/vsock_loopback.ll
llvm-dis-14
    vsock_diag.bc
    -o /tmp/vsock_diag.ll
llvm-dis-14
    vmw_vsock_virtio_transport.bc
    -o /tmp/virtio_transport.ll
llvm-dis-14
    vmw_vsock_virtio_transport_common.bc
    -o /tmp/virtio_common.ll
```

We had to do this to rename certain definitions that were repeated across modules. Specifically, the functions `init_module` and `cleanup_module` are defined in multiple modules among those we chose to merge. When considered as individual modules, they pose no problem, but once combined, they trigger an error due to the multiple definition of symbols. Since the names of the functions within the individual modules are arbitrary, we simply renamed them to make them unique. We provide just one example of the command used to do this, but the renamed

functions are located in the modules `vsock_loopback`, `vsock_diag`, `virtio_transport`, and `virtio_common`.

```
sed -i \
  -e 's/@init_module/@init_module_vsock_loopback/g' \
  -e 's/@cleanup_module/@cleanup_module_vsock_loopback/g' \
  /tmp/vsock_loopback.ll
...
```

After the modification, we reassemble the impacted modules to regenerate the bitcode:

```
llvm-as-14
    /tmp/vsock_loopback.ll
    -o vsock_loopback_renamed.bc
llvm-as-14
    /tmp/vsock_diag.ll
    -o vsock_diag_renamed.bc
llvm-as-14
    /tmp/virtio_transport.ll
    -o vmw_vsock_virtio_transport_renamed.bc
llvm-as-14
    /tmp/virtio_common.ll
    -o vmw_vsock_virtio_transport_common_renamed.bc
```

After that, we can proceed to link the modules to create a combined module, on which UAFX will be launched.

```
llvm-link \
    vsock.bc \
    vsock_loopback_renamed.bc \
    vsock_diag_renamed.bc \
    vmw_vsock_virtio_transport_renamed.bc \
    vmw_vsock_virtio_transport_common_renamed.bc \
    socket.bc \
    -o _vsock_all.bc
```

5.2 Test execution

The UAFX testing phase was conducted in two stages. As explained earlier, the UAFX paper explicitly states that reference counting issues have been ruled out. The first phase consisted of testing directly using the unmodified version of UAFX. In the second phase, we modified

the UAFX code to remove the constraints that rule out reference counting issues and repeated the tests on the same target bitcode.

5.2.1 Test with unmodified UAFX

As mentioned, the first test was conducted using UAFX as provided by the researchers, with controls in place to rule out active reference counting issues. We ran the script provided by the researchers on the unified bitcode:

```
./run_nohup.sh _bc/_vsock_all.bc results/_conf_uafx_vsock
```

Figure 11 represents an example of the output from running UAFX on the extracted bitcode.

```
=========================Main Analysis Phase (Callback)=========================
The identified callbacks to be analyzed:
[TIMING] Main Analysis Phase finished in : 1.900221e+00s
[MEM] Mem Statistics for Main Analysis Phase:
Mon Aug 17 22:57:58 2026

Memory Usage Statistics (in MB):
Min: 8.196094e+01
Max: 8.196094e+01
Average: 8.196094e+01
Median: 8.196094e+01
=========================Bug Detection Phase=========================
[TIMING] Bug Detection Phase Starts : Mon Aug 17 22:57:58 2026

-- detectUAFs(): _detectUAFS_ty1() for fo: 0x2a748230
getForwardEqvObjs(): for obj: 0x2a748230
getForwardEqvObjsOnce(): for obj: 0x2a748230
getForwardEqvObjs(): iteration: 0, src objs: 0x2a748230, new res objs: time spent (s): 1.966600e-05
getForwardEqvObjs(): done, the forward objs of 0x2a748230 are: 0x2a748230,
[TIMING] Bug Detection Phase finished in : 8.720200e-05s
[MEM] Mem Statistics for Bug Detection Phase:
Mon Aug 17 22:57:58 2026

Memory Usage Statistics (in MB):
Min: 8.196094e+01
Max: 8.196094e+01
Average: 8.196094e+01
Median: 8.196094e+01
All done: Mon Aug 17 22:57:58 2026

[MEM] Mem Statistics for All:
Mon Aug 17 22:57:58 2026

Memory Usage Statistics (in MB):
Min: 8.196094e+01
Max: 8.196094e+01
Average: 8.196094e+01
Median: 8.196094e+01
```

Figure 11: Execution results with unmodified UAFX.

As we can see, running UAFX on the aggregated bitcode completes successfully, finishing both

the Main Analysis Phase and the subsequent *Bug Detection* Phase, but the analysis does not produce any reports of potential use-after-free vulnerabilities.

Specifically, during the phase dedicated to callback analysis, the section titled "*The identified callbacks to be analyzed*" is empty, so UAFX hasn't identified any additional callbacks to be subjected to the interprocedural analysis.

During the Bug Detection Phase, a candidate object (`fo`) is considered, on which the `_detectUAFS_ty1()` procedure is executed. UAFX then attempts to determine its equivalent objects using `getForwardEqvObjs()`, but the propagation reaches a dead end as early as the first iteration: the resulting set contains only the starting object, and no additional equivalent objects are identified to which the analysis can be propagated.

Consequently, in this configuration, UAFX is unable to establish a connection between the candidate under consideration and a subsequent access that could be classified as a use-after-free. We can in fact see that the Bug Detection Phase ends in an extremely short amount of time (approximately 8.7×10^{-5} seconds) without generating any warnings.

This result does not prove the absence of the vulnerability in the analyzed code, but it indicates that, with the provided bitcode, entry functions, and using the standard UAFX configuration, the analysis failed to reconstruct a flow sufficient to trigger the UAF under investigation. We can assume that this could be caused by insufficient interprocedural propagation or the failure to resolve callbacks or indirect calls involved in the vulnerable path. Another possible cause could have been the explicitly mentioned exclusion of reference counting issues, for which we made a change to UAFX, as described below.

5.2.2 Test with modified UAFX

To modify UAFX, we traced the code execution flow starting from the entry point. Reference counting issues are filtered in `ModuleState.h` by calling the `_isWithRefcnt` function defined in `ModuleState.cpp` as follows:

```
bool _isWithRefcnt(InstLoc *floc, InstLoc *uloc) {
    // Heuristic 1: see whether the F is inside a refcnt
    // put function.
    static std::set<std::string> put_func_inc {
        "_put",
        "put_",
    };
    static std::set<std::string> get_func_inc {
        "_get",
        "get_",
    };
    if (floc) {
        std::vector<Instruction*> insts;
        if (dyn_cast<Instruction>(floc->inst)) {
            insts.push_back(
                dyn_cast<Instruction>(floc->inst)
            );
        }
        if (floc->ctx && !floc->ctx->empty()) {
            insts.insert(
                insts.begin(),
                floc->ctx->callSites->begin(),
                floc->ctx->callSites->end()
            );
        }
        for (int i = insts.size() - 1; i >= 0; --i) {
            Instruction *inst = insts[i];
            if (inst && inst->getParent() && inst->getFunction()) {
                std::string func = inst->getFunction()->getName().str();
                for (auto &s : put_func_inc) {
                    if (func.find(s) != std::string::npos) {
                        return true;
                    }
                }
            }
            // Also check the func name from the dbg info, which can
            // even reveal the names of the inlined functions.
            if (i % 2) {
                std::vector<std::string> fns;
                InstructionUtils::getHostFuncsFromDLoc(inst,fns);
                for (auto &fn : fns) {
                    for (auto &s : put_func_inc) {
                        if (fn.find(s) != std::string::npos) {
                            return true;
                        }
                    }
                }
            }
        }
    }
    // Comments about heuristics 2 and 3
    return false;
}
```

The function implements a lightweight heuristic to recognize cases in which a potential UAF is likely protected by a reference counting mechanism. It receives the free location and the use location, but in the current implementation it mainly reasons about the free site.

The first heuristic checks whether the free operation occurs inside, or through, a function whose name suggests a reference-count decrement/release operation. In particular, the analysis collects the instruction corresponding to the free site and the call sites in its calling context, then walks this sequence backwards and inspects the name of each enclosing function. If one of these function names contains patterns such as `_put` or `put_` (lines 4-7), the function assumes that the free may be guarded by a reference counting `put` operation and returns `true`.

The same check is also applied to function names recovered from debug information, which is useful because debug metadata can expose inlined function names that are not directly visible from the LLVM instruction's enclosing function and the heuristic can also detect cases where the reference counting operation was inlined.

The function also defines patterns for reference-count acquisition functions, such as `_get` and `get_` (lines 8-11), but these are not effectively used by the current heuristic.

At line 51, we omitted some comments from the original code [72] that mention possible future development of two additional heuristics:

- **Heuristic 2**: check whether the free site is dominated or post-dominated by an update to the reference count;
- **Heuristic 3**: check whether the usage site is protected by a change in the reference count.

At the time of writing, these heuristics have not yet been implemented, and therefore we have not taken them into account.

Overall, this is just a syntactic filter: it doesn't model the actual value of the reference counter, but instead it treats the presence of a likely reference-count release function in the free-side context as evidence that the reported UAF may be a false positive caused by imprecise reasoning about reference counting.

After making the change by disabling the check, we recompiled UAFX and relaunched it using the same bitcode as the execution with the checks enabled. However, the results are no different from the previous ones, as we can see in figure 12.
In this case as well, no callbacks are identified for analysis and, during the *Bug Detection Phase*, a single candidate object is considered. The search for equivalent objects in the forward direction immediately reaches a dead end, leaving only the initial object and failing to identify any further relationships useful for constructing a possible UAF path.

Removing the filter related to reference counting is therefore not sufficient, in this case, to reveal the vulnerability. This comparison also allows us to put into perspective the hypothesis formulated following the first execution: the absence of warnings does not appear to be

```
=========================Main Analysis Phase (Callback)=========================
The identified callbacks to be analyzed:
[TIMING] Main Analysis Phase finished in : 1.772787e+00s
[MEM] Mem Statistics for Main Analysis Phase:
Mon May  4 17:19:45 2026

Memory Usage Statistics (in MB):
Min: 8.153125e+01
Max: 8.153125e+01
Average: 8.153125e+01
Median: 8.153125e+01
=========================Bug Detection Phase=========================
[TIMING] Bug Detection Phase Starts : Mon May  4 17:19:45 2026

-- detectUAFs(): _detectUAFS_ty1() for fo: 0x1fda95d0
getForwardEqvObjs(): for obj: 0x1fda95d0
getForwardEqvObjsOnce(): for obj: 0x1fda95d0
getForwardEqvObjs(): iteration: 0, src objs: 0x1fda95d0, new res objs: time spent (s): 1.291500e-05
getForwardEqvObjs(): done, the forward objs of 0x1fda95d0 are: 0x1fda95d0,
[TIMING] Bug Detection Phase finished in : 5.223900e-05s
[MEM] Mem Statistics for Bug Detection Phase:
Mon May  4 17:19:45 2026

Memory Usage Statistics (in MB):
Min: 8.153125e+01
Max: 8.153125e+01
Average: 8.153125e+01
Median: 8.153125e+01
All done: Mon May  4 17:19:45 2026
```

Figure 12: Execution results with modified UAFX.

caused solely by the intentional exclusion of reference counting cases, since the behavior of the analysis remains virtually unchanged even without that filter.

The limitation appears to arise at an earlier or different stage of the analysis, for example in the identification and propagation of relationships between objects, or in the reconstruction of the interprocedural flow required to link the creation of an object to its subsequent use. These results, however, don't allow us to determine which of these mechanisms is responsible for the failure to detect the issue.

Chapter 6

Conclusions

The main objectives of this thesis were to reproduce and analyze a real-world use-after-free memory corruption vulnerability, CVE-2025-21756, affecting the Linux kernel's vsock subsystem, and to evaluate the efficacy of state-of-the-art static and dynamic analysis tools in detecting such reference counting bugs. Specifically, we evaluated CountDown (a refcount-guided dynamic fuzzer) and UAFX (a static analyzer) under various configurations to assess their ability to identify this class of vulnerabilities.

This chapter summarizes our experimental findings, discusses the underlying complexities that make reference counting vulnerabilities an open research problem, addresses the limitations of our evaluation environment and outlines promising directions for future work.

6.1 Summary of experimental findings

Our experimental study revealed that detecting reference-count-driven UAF vulnerabilities remains a challenging task for automated tools, even when these tools are modified or guided with prior knowledge of the target vulnerability:

1. **Dynamic Analysis with CountDown**: in the unguided fuzzing campaign (without a starting corpus), CountDown ran for several days but did not produce any crash reports or trace any anomalous reference counting behaviors related to CVE-2025-21756.

   In the guided fuzzing campaign, we provided CountDown with a highly targeted seed corpus translating the maintainers' test case into Syzkaller's typed syscall format and restricted the active syscall set to vsock interfaces. Over a multi-day campaign executing more than 1.7 million test cases, CountDown still failed to trigger a crash or record any anomalous kref history, confirming that even when primed with a valid trigger sequence, coverage-guided and refcount-guided fuzzers struggle to recreate the precise execution state and satisfy the strict argument dependencies necessary to expose this class of bugs under realistic workloads.

2. **Static analysis with UAFX**: the unmodified version of UAFX failed to detect the vulnerability in the consolidated `_vsock_all.bc` bitcode, completing its bug detection phase in less than a millisecond without generating any warnings or identifying any relevant callback structures.

   To investigate whether this was solely due to UAFX's intentional exclusion of reference counting patterns, we modified UAFX by disabling the syntactic filter `_isWithRefcnt` in `ModuleState.cpp`. Despite removing this constraint, the analysis behavior remained virtually identical, failing to reconstruct any UAF path. This demonstrated that the tool's blind spot lies deeper within its core static analysis mechanisms (interprocedural propagation, alias analysis, handling of complex indirect function calls and kernel-specific callbacks).

   One reason for this could also be related to how UAFX itself works: since the paper tests it only on drivers, there is no guarantee that it will be equally effective when applied to a set of different modules working together.

6.2 The complexity of reference counting UAFs as an open problem

The fact that both a cutting-edge static analyzer and a specialized refcount fuzzer failed to identify CVE-2025-21756 underscores that reference-count-related memory safety issues represent a highly complex and still open problem in kernel security. Reference-counting bugs in the kernel affect several areas:

1. **Non-linear execution paths**: as shown in CVE-2025-21756, the bug is "cross-entry", involving distinct syscalls (socket, connect, and bind) executing across different entry points and contexts;

2. **State-dependent preconditions**: triggering this specific vulnerability requires establishing a highly complex state by exhausting the host's available automatic vsock ports via a sequence of 24 consecutive bindings, which steers the first connection attempt into a failure path that leaves the socket in an unbound state but with an active transport layer. Forcing these precise state-dependent constraints is something that static engines cannot easily model symbolically (due to path explosion and environment modeling limits) and that dynamic fuzzers cannot easily hit by chance, even when guided by code coverage or simple refcount changes;

3. **Semantic vs. syntactic analysis**: UAFX relies on simple syntactic heuristics (i.e. matching function names containing "put" or "get") to filter out refcount candidates, rather than performing deep semantic reasoning on the counter's value. Conversely, dynamic

fuzzers are bound by the temporal gap between the reference count drop and the actual memory access; if the allocator's state or the timing is not perfectly aligned, the UAF will not result in an immediate, detectable crash, leaving the bug silent unless captured by heavy sanitizers like KASAN under specific heap configurations, which is probably what the researcher(s) who first discovered the bug did.

6.3 Evaluation constraints and limitations

An analysis of our experimental evaluation must highlight several structural and environmental limitations, which we list below.

Hardware and computational limits

Dynamic fuzzing with CountDown is an exceptionally resource-intensive process. We allocated 6 parallel virtual machine instances (totaling 12 vCPUs and 12 GB of RAM) to the fuzzer. While this configuration represents a reasonable academic setup, it is modest compared to industrial-scale fuzzing farms that utilize hundreds of virtual machines running continuously. For example, the test configuration used by the researchers who authored the CountDown paper [9] consisted of a setup of 16 virtual machines, each with 2 CPUs, for a total of 3 days of execution.

Because CountDown constantly monitors and logs every single reference-counting operation across the kernel, it introduces a significant instrumentation overhead. Under our hardware limits, this overhead directly constrained the execution throughput (number of executions per second), making it harder to explore highly deep or improbable paths.

Search space constraints

Fuzzing is inherently non-deterministic and requires massive time scales to cover deep state spaces. Although our campaign ran for several days and reached over 1.7 million executions, the combinatorial search space of vsock socket operations, especially with the strict requirement of consecutive port exhaustion, remained too vast to be exhausted. A longer fuzzing window or a more powerful hardware environment might have yielded different results.

6.4 Promising directions for future work

The results and limitations of this study suggest several directions for future research on use-after-free vulnerabilities and, more specifically, on reference-counting errors in the Linux kernel.

Broader characterization of recent UAF vulnerabilities

A first direction concerns a broader and more systematic study of recent use-after-free vulnerabilities affecting the Linux kernel. The vulnerabilities considered in this thesis provide useful examples of different lifetime errors, but they represent only a limited portion of the UAF vulnerabilities discovered in recent kernel versions.

Extending the analysis to a larger and more recent set of vulnerabilities could help identify recurring patterns in their root causes, affected subsystems or triggering conditions. Such a study could also determine how frequently reference-counting errors occur compared with other causes of use-after-free vulnerabilities [6] and whether particular classes of lifetime errors are becoming more or less relevant as the kernel and its defensive mechanisms evolve.

Systematic evaluation of dynamic analysis tools

A second direction is the systematic evaluation of dynamic analysis techniques against a consolidated ground truth of known use-after-free vulnerabilities. Evaluating different approaches on the same set of reproducible vulnerabilities would provide a more reliable basis for understanding their actual detection coverage.

This should consider not only whether a vulnerability can eventually be detected, but also whether the execution strategy is capable of reaching the conditions required to expose it. This distinction is particularly important for vulnerabilities whose manifestation depends on specific conditions (for example concurrent instructions) or uncommon execution paths. An example of this need is explained in section 4.3, where we highlight the impossibility of following a certain flow, especially in terms of arbitrary inputs, due to the very nature of the fuzzer. In fact, to guide the execution more precisely, it would be necessary to modify the very essence of the tool by introducing a way to "fix" parameters that remain unchanged during executions. Realistically, this can be useful in a situation where it is already known that a specific execution flow may cause certain problems, but it is not known exactly which ones, and one wishes to explore that direction.

A sufficiently broad and reproducible benchmark could therefore provide a clearer characterization of the strengths and limitations of existing dynamic approaches and make comparisons between different techniques more meaningful.

Static analysis of reference-counting correctness

Finally, reference counting represents a particularly relevant direction for further static analysis research. As shown throughout this thesis, determining whether an object is released while references to it are still logically active requires reasoning about object ownership and lifetime across potentially complex execution paths.

Future work could investigate static approaches specifically aimed at identifying inconsistencies between reference acquisition, propagation and uses, and release. The objective would

be to determine whether reference counting invariants can be reconstructed and verified without relying on the vulnerability being dynamically triggered.

A more general investigation of this problem could help establish which classes of reference counting errors can be effectively detected statically, which sources of imprecision limit current approaches and where static analysis must be complemented by dynamic techniques. This would also provide a basis for evaluating whether reference counting vulnerabilities require specialized analysis or can be adequately addressed within more general approaches to use-after-free detection.

Bibliography

[1] Alex Rebert and Christoph Kern. Secure by design: Google's perspective on memory safety, 2024. URL https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety.

[2] Oscar Llorente-Vazquez, Igor Santos-Grueiro, Iker Pastor-Lopez, and Pablo Garcia Bringas. Detection, exploitation and mitigation of memory errors. *Logic Journal of the IGPL*, 32(2):281–292, 2024. doi: 10.1093/jigpal/jzae008. URL https://doi.org/10.1093/jigpal/jzae008.

[3] Microsoft Security Response Center. A proactive approach to more secure code, 2019. URL https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code/.

[4] The Linux Kernel Developers. Linux kernel coding style, 2026. URL https://www.kernel.org/doc/html/latest/process/coding-style.html. Section 11: Data structures.

[5] Corey Minyard and Thomas Hellstrom. Adding reference counters (krefs) to kernel objects. The Linux Kernel Documentation, 2026. URL https://www.kernel.org/doc/html/latest/core-api/kref.html.

[6] The MITRE Corporation. Cwe-416: Use after free, 2006. URL https://cwe.mitre.org/data/definitions/416.html.

[7] National Institute of Standards and Technology. Cve-2025-21756 - national vulnerability database, 2025. URL https://nvd.nist.gov/vuln/detail/CVE-2025-21756.

[8] Stefano Garzarella. Vsock: Vm ⟷ host socket with minimal configuration - presentation at devconf.cz 2020, 2020. URL https://static.sched.com/hosted_files/devconfcz2020a/b1/DevConf.CZ_2020_vsock_v1.1.pdf.

[9] Shuangpeng Bai, Zhechang Zhang, and Hong Hu. Countdown: Refcount-guided fuzzing for exposing temporal memory errors in linux kernel, 2024. URL https://dl.acm.org/doi/epdf/10.1145/3658644.3690320.

[10] Hang Zhang, Jangha Kim, Chuhong Yuan, Zhiyun Qian, and Taesoo Kim. Statically discover cross-entry use-after-free vulnerabilities in the linux kernel, 2025. URL https://www.ndss-symposium.org/wp-content/uploads/2025-559-paper.pdf.

[11] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation, 2004. URL https://dl.acm.org/doi/10.5555/977395.977673.

[12] GNU Project. The gnu allocator, 2026. URL https://sourceware.org/glibc/manual/2.27/html_node/The-GNU-Allocator.html.

[13] GNU Project. Efficiency considerations for malloc, 2026. URL https://sourceware.org/glibc/manual/2.23/html_node/Efficiency-and-Malloc.html.

[14] Wenzhe Zhang. Efficient detection of dangling pointer error for c/c++ programs, 2017. URL https://www.researchgate.net/publication/319596347_Efficient_detection_of_dangling_pointer_error_for_CC_programs.

[15] Niels Provos, Markus Friedl, and Peter Honeyman. Preventing privilege escalation, 2003. URL https://www.academia.edu/1960780/Preventing_privilege_escalation.

[16] George E. Collins. A method for overlapping and erasure of lists, 1960. URL https://dl.acm.org/doi/10.1145/367487.367501.

[17] Aleph One. Smashing the stack for fun and profit, 1996. URL https://www.andrew.cmu.edu/course/18-330/2025f/reading/One_1996_Smashing%20The%20Stack%20For%20Fun%20And%20Profit.pdf.

[18] Peter Breuer, Simon Pickin, and María Mercedes Larrondo Petrie. Detecting deadlock, double-free and other abuses in a million lines of linux kernel source, 2006. URL https://www.academia.edu/1413564/Detecting_deadlock_double_free_and_other_abuses_in_a_million_lines_of_linux_kernel_source.

[19] Alexander Sotirov. Heap feng shui in javascript, 2007. URL https://blackhat.com/presentations/bh-usa-07/Sotirov/Whitepaper/bh-usa-07-sotirov-WP.pdf.

[20] Hovav Shacham, Matthew Page, Ben Pfaff, Eu-Jin Goh, Nagendra Modadugu, and Dan Boneh. On the effectiveness of address-space randomization, 2004. URL https://www.andrew.cmu.edu/course/18-330/2025f/reading/Shacham_2004_ASLR_Effectiveness.pdf.

[21] Martín Abadi, Mihai Budiu, Úlfar Erlingsson, and Jay Ligatti. Control-flow integrity principles, implementations, and applications, 2009. URL https://dl.acm.org/doi/10.1145/1609956.1609960.

[22] Xingyu Jin. The quantum state of linux kernel garbage collection cve-2021-0920 (part i), 2022. URL https://projectzero.google/2022/08/the-quantum-state-of-linux-kernel.html.

[23] Michael Hoefler. Linux kernel exploitation - cve-2025-21756: Attack of the vsock, 2025. URL https://hoefler.dev/articles/vsock.html.

[24] Audrey. Linux market share statistics 2026: Desktop, server and enterprise adoption trends, 2026. URL https://fosspost.org/linux-market-share-statistics/.

[25] Microsoft. Kernel object in microsoft windows docs, 2026. URL https://learn.microsoft.com/en-us/windows/win32/sysinfo/kernel-objects.

[26] Linux kernel developers. sched.h in linux kernel, 2026. URL https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/sched.h.

[27] Linux kernel developers. struct socket in linux kernel, 2026. URL https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/net.h#L127.

[28] Linux kernel developers. struct sock in linux kernel, 2026. URL https://elixir.bootlin.com/linux/v7.1.8/source/include/net/sock.h#L244.

[29] Linux kernel developers. struct inode in linux kernel, 2026. URL https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/fs.h#L767.

[30] Linux kernel developers. struct sk_buff in linux kernel, 2026. URL https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/skbuff.h#L775.

[31] Jeff Bonwick. The slab allocator: An object-caching kernel memory allocator, 1994. URL https://people.eecs.berkeley.edu/~kubitron/courses/cs194-24-S14/hand-outs/bonwick_slab.pdf.

[32] Jonathan Corbet. The slub allocator, 2007. URL https://lwn.net/Articles/229984/.

[33] LLVM Project. Clang static analyzer, 2008. URL https://github.com/llvm/llvm-project/blob/main/clang/lib/StaticAnalyzer/README.txt.

[34] Cristiano Calcagno and Dino Distefano. Infer: An automatic program verifier for memory safety of c programs, 2011. URL https://scispace.com/pdf/infer-an-automatic-program-verifier-for-memory-safety-of-c-1d048xu12f.pdf.

[35] syzkaller project. Syzkaller: unsupervised coverage-guided kernel fuzzer, 2015. URL https://github.com/google/syzkaller/blob/master/docs/internals.md.

[36] GNU Project. Gnu compiler collection (gcc), 1987. URL https://github.com/gcc-mirror/gcc.

[37] Hua Yan, Yulei Sui, Shiping Chen, and Jingling Xue. Spatio-temporal context reduction: A pointer-analysis-basedstatic approach for detecting use-after-free vulnerabilities, 2018. URL https://dl.acm.org/doi/epdf/10.1145/3180155.3180178.

[38] Cppcheck project. Cppcheck: static analysis of c/c++ code, 2007. URL https://github.com/cppcheck-opensource/cppcheck.

[39] SVF Project. Svf: Static value-flow analysis framework for source code, 2016. URL https://github.com/SVF-tools/SVF.

[40] Liang He, Hong Hu, Purui Su, Yan Cai, and Zhenkai Liang. Freewill: Automatically diagnosing use-after-free bugs via reference miscounting detection on binaries, 2022. URL https://www.usenix.org/system/files/sec22-he-liang.pdf.

[41] Jian Liu, Lin Yi, Weiteng Chen, Chengyu Song, Zhiyun Qian, and Qiuping Yi. Linkrid: Vetting imbalance reference counting in linux kernel with symbolic execution, 2022. URL https://www.usenix.org/system/files/sec22-liu-jian.pdf.

[42] Jangseop Shin, Donghyun Kwon, Jiwon Seo, Yeongpil Cho, and Yunheung Paek. Crcount: Pointer invalidation with reference counting to mitigate use-after-free in legacy c/c++, 2019. URL https://www.ndss-symposium.org/wp-content/uploads/2019/02/ndss2019_05A-4_Shin_paper.pdf.

[43] Jiayi Ye, Chao Zhang, and Xinhui Han. Uafchecker: Scalable static detection ofuse-after-free vulnerabilities, 2014. URL https://dl.acm.org/doi/epdf/10.1145/2660267.2662394.

[44] Jia-Ju Bai, Julia Lawall, Qiu-Liang Chen, and Shi-Min Hu. Effective static analysis of concurrency use-after-free bugs in linux device drivers, 2019. URL https://www.usenix.org/system/files/atc19-bai.pdf.

[45] Pan Bian, Bin Liang, Jianjun Huang, Wenchang Shi, Xidong Wang, and Jian Zhang. Sinkfinder: Harvesting hundreds of unknown interestingfunction pairs with just one seed, 2020. URL https://dl.acm.org/doi/epdf/10.1145/3368089.3409678.

[46] Yunlong Lyu, Yi Fang, Yiwei Zhang, Qibin Sun, Siqi Ma, Elisa Bertino, Kangjie Lu, and Juanru Li. Goshawk: Hunting memory corruptions via structure-aware and object-centric memory operation synopsis, 2022. URL https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9833613.

[47] Xin Tan, Yuan Zhang, Xiyu Yang, Kangjie Lu, and Min Yang. Detecting kernel refcount bugs with two-dimensional consistency checking, 2021. URL https://yuanxzhang.github.io/paper/cid-security21.pdf.

[48] Leonardo de Moura and Nikolaj Bjørner. Z3: an efficient smt solver, 2008. URL https://dl.acm.org/doi/10.5555/1792734.1792766.

[49] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. kvm : the linux virtual machine monitor, 2007. URL https://api.semanticscholar.org/CorpusID:2408450.

[50] Fabrice Bellard. Qemu, a fast and portable dynamic translator, 2005. URL https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator.

[51] Edouard Bugnion, Scott Devine, and Mendel Rosenblum. Disco: Running commodity operating systems on scalable multiprocessors - disco would later become vmware, 1997. URL https://pages.cs.wisc.edu/~remzi/Classes/838/Spring2013/Papers/bugnion97disco.pdf.

[52] Microsoft. Hyper-v architecture - microsoft learn, 2008. URL https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/architecture.

[53] Eugenio Pérez Martín. Packed virtqueue: How to reduce overhead with virtio, 2020. URL https://www.redhat.com/en/blog/packed-virtqueue-how-reduce-overhead-virtio.

[54] Linux kernel developers. af_vsock module, linux kernel code, v6.6.78, 2025. URL https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/vmw_vsock/af_vsock.c?h=v6.6.78.

[55] Linux Test Project. Cve-2025-21756 test case in linux test project release 20250930, 2025. URL https://github.com/linux-test-project/ltp/releases/tag/20250930.

[56] Linux kernel developers. af_vsock module, linux kernel code, v6.6.78, line 167, 2025. URL https://elixir.bootlin.com/linux/v6.6.78/source/net/vmw_vsock/af_vsock.c#L167.

[57] Michal Luczaj. vsock: Keep the binding until socket destruction, linux kernel patch, 2025. URL https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=fcdd2242c0231032fc84e1404315c245ae56322a.

[58] Fedora Project. Virtualization – getting started (fedora), 2026. URL https://docs.fedoraproject.org/en-US/quick-docs/virtualization-getting-started/.

[59] Shuangpeng Bai, Zhechang Zhang, and Hong Hu. Countdown repository on github, 2024. URL https://github.com/psu-security-universe/countdown.

[60] syzkaller project. Linux kernel configs on syzkaller's github repository, 2026. URL https://github.com/google/syzkaller/blob/master/docs/linux/kernel_configs.md.

[61] Linux Kernel Driver DataBase (LKDDb). Config_init_stack_none: no automatic stack variable initialization (weakest), 2026. URL https://cateee.net/lkddb/web-lkddb/INIT_STACK_NONE.html.

[62] Linux Kernel Driver DataBase (LKDDb). Config_init_on_alloc_default_on: Enable heap memory zeroing on allocation by default, 2026. URL https://cateee.net/lkddb/web-lkddb/INIT_ON_ALLOC_DEFAULT_ON.html.

[63] Linux Kernel Driver DataBase (LKDDb). Config_init_on_free_default_on: Enable heap memory zeroing on free by default, 2026. URL https://cateee.net/lkddb/web-lkddb/INIT_ON_FREE_DEFAULT_ON.html.

[64] Debian Project. Debootstrap documentation on debian wiki, 2026. URL https://wiki.debian.org/Debootstrap.

[65] syzkaller project. Syzkaller coverage on github, 2026. URL https://github.com/google/syzkaller/blob/master/docs/coverage.md.

[66] kernelconfig.io. Kernel memory leak detector, 2026. URL https://www.kernelconfig.io/CONFIG_DEBUG_KMEMLEAK.

[67] Linux kernel developers. Kernel address sanitizer (kasan), 2026. URL https://docs.kernel.org/dev-tools/kasan.html.

[68] Clark Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. Satisfiability modulo theories, 2009. URL https://people.eecs.berkeley.edu/~sseshia/pubdir/SMT-BookChapter2e.pdf.

[69] gllvm project. gllvm documentation - github, 2019. URL https://jenniniku.github.io/gllvm/.

[70] WLLVM project. Wllvm - a wrapper script to build whole-program llvm bitcode files - github, 2011. URL https://github.com/travitch/whole-program-llvm.

[71] Linux man-pages project. Syscalls in linux manual page, 2026. URL https://man7.org/linux/man-pages/man2/syscall.2.html.

[72] UAFX Project. Modulestate6.cpp in uafx's github repository, 2025. URL https://github.com/uafx/uafx/blob/main/llvm_analysis/MainAnalysisPasses/SoundyAliasAnalysis/src/ModuleState.cpp.

Project developed at the System security and cryptography Lab (LaSER)
https://security.di.unimi.it/